PopSystem Development Guide v1.38

Development Planning - NewPOPSys v1.38

> **Purpose**: Agile development management artifacts complementing the IEEE 830 SRS in `07_SRS/` **Methodology**: Hybrid Agile/Scrum with Kanban + Gantt tracking **Timeline**: 13 weeks to Beta (End Q1 2026) **Budget**: $150,000

Quick Navigation

Section	Purpose	Key Documents
00_Meta	Framework & standards	DEV_PLANNING_FRAMEWORK.md
01_Master_Plan	Strategic planning	MASTER_DEVELOPMENT_PLAN.md
02_Sprint_Roadmap	Sprint breakdown	SPRINT_ROADMAP.md
03_Technical_Specs	Implementation specs	AI_DEV_SPECS.md
04_Tracking_Artifacts	Kanban & Gantt	GANTT_CHART.md
05_Agile_Suite	DoD, DoR, checklists	DEFINITION_OF_DONE.md
06_Risk_Quality	Risk & quality gates	RISK_REGISTER.md
07_SRS_Alignment	SRS sync tracking	SRS_SYNC_STATUS.md
99_Templates	Reusable templates	Sprint, Task, Retro

Current Sprint Status

```
Sprint: S0 - Foundation          Status: NOT STARTED
Start:  2026-01-06               End: 2026-01-17
```

Metric	Value
Tasks Total	12

Completed	0
In Progress	0
Blocked	0

See CURRENT_SPRINT.md for details.

Sprint Timeline

```
Week 1-2   : S0 - Foundation & Infrastructure
Week 3-4   : S1 - Authentication & Core UI
Week 5-6   : S2 - Mobile App Core
Week 7-8   : S3 - Brand Admin Portal
Week 9-10  : S4 - PSP Operations Portal
Week 11-12 : S5 - Store Portal & Integration
Week 13    : S6 - Polish, Testing & Beta Launch
```

See GANTT_CHART.md for visual timeline.

Relationship to SRS

This folder contains **development process** artifacts. Requirements are in `07_SRS/`:

This Folder (08_Dev_Planning)	SRS Folder (07_SRS)
How we build	What we build
Sprint plans	Screen specifications
Task tracking	Functional requirements
Kanban/Gantt	Use cases & workflows
Quality gates	Acceptance criteria

Cross-reference: SRS_SYNC_STATUS.md

Key Artifacts

For Daily Standups

- CURRENT_SPRINT.md - Today's tasks
- SPRINT_BOARD_VIEW.md - Kanban view

For Sprint Planning

- BACKLOG.md - Prioritized backlog
- DEFINITION_OF_READY.md - Entry criteria

For Sprint Review

- BURNDOWN_CHART.md - Progress tracking
- SPRINT_METRICS.md - Velocity data

For Releases

- RELEASE_CHECKLIST.md - Pre-release verification
- QUALITY_GATES.md - Quality checkpoints

Document Index

00_Meta/ (Framework)

- `DEV_PLANNING_FRAMEWORK.md` - Methodology documentation
- `NAMING_CONVENTIONS.md` - Standards for task IDs, branches
- `RELATIONSHIP_DIAGRAM.md` - How documents connect

01_Master_Plan/ (Strategy)

- `MASTER_DEVELOPMENT_PLAN.md` - Project overview, budget, scope
- `PROJECT_CHARTER.md` - Executive summary
- `MILESTONE_SCHEDULE.md` - High-level timeline with gates
- `RESOURCE_ALLOCATION.md` - Team structure

02_Sprint_Roadmap/ (Sprints)

- `SPRINT_ROADMAP.md` - Sprint overview
- `SPRINT_CALENDAR.md` - Date-bound schedule
- `DEPENDENCY_MAP.md` - Cross-sprint dependencies
- `Sprints/SPRINT_S0_Foundation.md` through `SPRINT_S6_Launch.md`

03_Technical_Specs/ (Implementation)

- `AI_DEVELOPMENT_HARNESS.md` - **NEW**: Agentic workflow & harness strategy
- `AI_DEV_SPECS.md` - Detailed technical specs & code patterns
- `TECH_STACK_DECISIONS.md` - ADR-style decisions
- `ARCHITECTURE_OVERVIEW.md` - System architecture
- `MONOREPO_STRUCTURE.md` - Code organization
- `IMPLEMENTATION_PATTERNS.md` - Code patterns

04_Tracking_Artifacts/ (Progress)

- `Sprint_Board/` - Kanban views
- `Gantt_Timeline/` - Timeline views
- `Metrics/` - Burndown, velocity, flow

05_Agile_Suite/ (Process)

- `DEFINITION_OF_DONE.md` - Quality criteria
- `DEFINITION_OF_READY.md` - Entry criteria
- `ACCEPTANCE_CRITERIA_GUIDE.md` - AC writing guide
- `TECH_DEBT_REGISTER.md` - Technical debt tracking
- `RELEASE_CHECKLIST.md` - Release verification
- `TESTING_STRATEGY.md` - Test approach
- `RETROSPECTIVE_LOG.md` - Retro summaries

06_Risk_Quality/ (Governance)

- `RISK_REGISTER.md` - Active risks
- `QUALITY_GATES.md` - Sprint checkpoints
- `DEPENDENCY_RISKS.md` - External dependencies
- `ISSUE_TRACKER_POLICY.md` - Bug triage

07_SRS_Alignment/ (Sync)

- `SRS_SYNC_STATUS.md` - Alignment tracker
- `SCREEN_IMPLEMENTATION_MAP.md` - Screen to sprint mapping
- `SUPP_IMPLEMENTATION_MAP.md` - SUPP to sprint mapping
- `CHANGE_CONTROL_LOG.md` - Scope changes

99_Templates/ (Reusable)

- `SPRINT_TEMPLATE.md`
- `TASK_TEMPLATE.md`
- `RETROSPECTIVE_TEMPLATE.md`
- `RELEASE_NOTES_TEMPLATE.md`
- `INCIDENT_TEMPLATE.md`

Created: 2026-01-01 | Last Updated: 2026-01-01

Master Development Plan

NewPOPSys v1.38 - PSP-led POP Campaign Orchestration Platform

---

Executive Summary

NewPOPSys v1.38 represents a complete modernization of the POP (Point of Purchase) campaign management system, transitioning from the legacy Ninja platform to a PSP-led orchestration model. This document serves as the authoritative development plan, outlining budget allocation, timeline, technical architecture, and deliverables.

Project Overview

Attribute	Value
Project Name	NewPOPSys v1.38
Project Type	PSP-led POP Campaign Orchestration Platform
Development Model	AI-Assisted Agile Development
Target Release	Beta - End Q1 2026
Total Interfaces	81 (24 Primary + 6 Additional + 51 Modals)

Budget Allocation

Total Budget: $150,000

Category	Percentage	Amount	Description
Development	60%	$90,000	Frontend, Backend, API, Integration
Infrastructure	15%	$22,500	Cloud hosting, CI/CD, DevOps
Quality Assurance	10%	$15,000	Testing, automation, UAT
Design	10%	$15,000	UI/UX, prototyping, design system
Contingency	5%	$7,500	Risk mitigation, scope changes

Budget Distribution by Sprint

Sprint	Duration	Budget Allocation	Cumulative
S0	2 weeks	$18,000 (12%)	$18,000
S1	2 weeks	$22,500 (15%)	$40,500
S2	2 weeks	$27,000 (18%)	$67,500
S3	2 weeks	$27,000 (18%)	$94,500
S4	2 weeks	$27,000 (18%)	$121,500
S5	2 weeks	$21,000 (14%)	$142,500
S6	1 week	$7,500 (5%)	$150,000

Timeline

Project Duration: 13 Weeks to Beta (End Q1 2026)

```
Week 1-2   | S0: Foundation & Setup
Week 3-4   | S1: Authentication & Core UI
Week 5-6   | S2: Mobile App Development
Week 7-8   | S3: Brand Admin Portal
Week 9-10  | S4: PSP Operations Center
Week 11-12 | S5: Store Portal
Week 13    | S6: Integration & Beta Launch
```

Sprint Structure

Sprint	Name	Duration	Start	End	Focus Area
S0	Foundation	2 weeks	Week 1	Week 2	Infrastructure, Architecture, Design System
S1	Auth & Core	2 weeks	Week 3	Week 4	Authentication, Navigation, Core Components
S2	Mobile App	2 weeks	Week 5	Week 6	PWA, Field Rep Features, Offline Support
S3	Brand Admin	2 weeks	Week 7	Week 8	Campaign Management, Asset Library, Reporting
S4	PSP Ops	2 weeks	Week 9	Week 10	Production Queue, Scheduling, Fulfillment
S5	Store Portal	2 weeks	Week 11	Week 12	Store Management, Inventory, Requests

S6	Beta Launch	1 week	Week 13	Week 13	Integration Testing, Deployment, Beta Release

Cross-Reference: See Sprint Roadmap for detailed sprint breakdowns.

Technology Stack

Frontend Architecture

Layer	Technology	Version	Purpose
Framework	React	18.x	Component-based UI development
Language	TypeScript	5.x	Type-safe development
State Management	Zustand	4.x	Lightweight state management
Styling	Tailwind CSS	3.x	Utility-first CSS framework
Build Tool	Vite	5.x	Fast development and bundling
Monorepo	Turborepo	Latest	Monorepo build system

Backend Architecture

Layer	Technology	Version	Purpose
Runtime	Node.js	20.x LTS	Server-side JavaScript
Framework	Fastify	4.x	High-performance HTTP framework
ORM	Drizzle ORM	Latest	Type-safe database queries
Database	PostgreSQL	16.x	Primary data store
Cache	Redis	7.x	Session management, caching
Queue	BullMQ	5.x	Background job processing

Infrastructure

Component	Technology	Purpose
Container	Docker	Application containerization
Orchestration	Docker Compose	Local development orchestration
CI/CD	GitHub Actions	Automated testing and deployment
CDN	Cloudflare	Asset delivery, edge caching

Monitoring	Grafana + Loki	Observability and logging

Interface Inventory

Total: 81 Interfaces

Primary Screens (24)

Module	Count	Screens
Mobile App	8	Dashboard, Campaign List, Task Detail, Photo Capture, Survey, Route, Profile, Sync
Brand Admin	6	Dashboard, Campaigns, Assets, Stores, Reports, Settings
PSP Operations	6	Queue, Schedule, Production, Shipping, Analytics, Config
Store Portal	4	Dashboard, Inventory, Requests, History

Additional Screens (6)

Screen	Purpose
Login	Universal authentication
Registration	User onboarding
Password Reset	Account recovery
Onboarding	First-time user flow
Help Center	Support documentation
Notifications	System alerts management

Modal Components (51)

Category	Count	Examples
Form Modals	18	Create Campaign, Add Asset, New Store
Confirmation Dialogs	12	Delete, Submit, Approve, Reject
Detail Views	10	Campaign Detail, Task Detail, Order Detail
Selection Modals	6	Store Picker, Date Range, Filter Options
Media Modals	5	Image Preview, Gallery, Photo Annotation

Cross-Reference: See Sprint Roadmap for interface delivery schedule.

Quality Gates

Code Quality Standards

Metric	Target	Tool
Code Coverage	≥ 80%	Jest + Vitest
Type Coverage	100%	TypeScript strict mode
Lint Score	0 errors	ESLint + Prettier
Bundle Size	< 200KB initial	Bundlesize
Lighthouse Score	≥ 90	Chrome DevTools

Performance Targets

Metric	Target	Measurement
First Contentful Paint	< 1.5s	Lighthouse
Time to Interactive	< 3.0s	Lighthouse
API Response Time	< 200ms (p95)	Application Monitoring
Database Query Time	< 50ms (p95)	Query Logging

Risk Management

Identified Risks

Risk	Probability	Impact	Mitigation
Scope creep	Medium	High	Strict change control, sprint boundaries
Technical debt	Medium	Medium	Code reviews, refactoring sprints
Integration delays	Low	High	Early API contracts, mock services
Resource availability	Low	Medium	Cross-training, documentation
Third-party dependencies	Low	Medium	Vendor evaluation, fallback options

---

Success Metrics

Launch Criteria

- [ ] All 81 interfaces implemented and tested
- [ ] 80% code coverage achieved
- [ ] Zero critical/high severity bugs
- [ ] Performance targets met
- [ ] Security audit passed
- [ ] UAT sign-off received

Post-Launch KPIs

KPI	Target	Timeframe
User Adoption	500 active users	30 days post-launch
System Uptime	99.5%	Monthly
User Satisfaction	≥ 4.0/5.0	Quarterly survey
Bug Resolution	< 24h critical	Ongoing

Document References

Document	Location	Purpose
Project Charter	PROJECT_CHARTER.md	Executive summary and vision
Milestone Schedule	MILESTONE_SCHEDULE.md	High-level gates and dates
Resource Allocation	RESOURCE_ALLOCATION.md	Team structure and roles
Sprint Roadmap	../02_Sprint_Roadmap/	Detailed sprint planning

---

Approval

Role	Name	Date	Signature
Project Sponsor			
Technical Lead			
Product Owner			

QA Lead			

Last Updated: 2026-01-01

POP System Sprint Roadmap

Overview

This document provides a comprehensive overview of the 6-sprint development roadmap for the POP System. The project spans 13 weeks, transforming the vision into a production-ready platform for Point-of-Purchase campaign management.

Sprint Summary

Sprint	Theme	Duration	Key Deliverables
S0	Foundation & Infrastructure	Week 1-2	Monorepo, Auth Foundation, Database, CI/CD
S1	Authentication & Core UI	Week 3-4	Universal Login, MFA, Core Layouts, Role Routing
S2	Mobile App Core	Week 5-6	Store Dashboard, Campaigns, Photo Capture, Checklists
S3	Brand Admin Portal	Week 7-8	Brand Dashboard, Campaign Management, Analytics
S4	PSP Operations Portal	Week 9-10	PSP Dashboard, Campaign Oversight, Shipments
S5	Store Portal & Integration	Week 11-12	Store Portal, Calendar, Display Compliance
S6	Polish, Testing & Beta Launch	Week 13	E2E Testing, Performance, Beta Release

---

Sprint Themes

S0: Foundation & Infrastructure (Week 1-2)

Goal: Establish the technical foundation that all subsequent development will build upon.

Key Objectives:

- Set up Turborepo monorepo structure with shared packages
- Implement authentication foundation with Supabase
- Configure PostgreSQL database with initial schema
- Establish CI/CD pipelines with GitHub Actions

- Configure development, staging, and production environments

Success Metrics:

- All team members can run local development environment
- CI pipeline passes on all commits
- Database migrations run successfully
- Authentication flow works end-to-end

S1: Authentication & Core UI (Week 3-4)

Goal: Deliver a polished, secure authentication experience and establish core UI patterns.

Key Objectives:

- Universal login page with role-based redirects
- Multi-factor authentication (MFA) implementation
- Core layout components (headers, navigation, sidebars)
- Role-based routing and access control
- Design system implementation with Tailwind CSS

Success Metrics:

- Users can log in with email/password and MFA
- Correct portal loaded based on user role
- All core UI components documented in Storybook
- 100% accessibility compliance (WCAG 2.1 AA)

S2: Mobile App Core (Week 5-6)

Goal: Build the essential mobile app functionality for store associates.

Key Objectives:

- Store dashboard with campaign overview
- Campaign list with status indicators
- Photo capture with AI-powered quality validation
- Checklist execution with offline support
- Push notification integration

Success Metrics:

- Store associates can view assigned campaigns
- Photos captured meet quality requirements
- Checklists work offline and sync when online
- Push notifications delivered within 30 seconds

S3: Brand Admin Portal (Week 7-8)

Goal: Empower brand administrators to manage campaigns effectively.

Key Objectives:

- Brand dashboard with key metrics
- Campaign creation and management workflow
- Store selection with filtering and bulk actions
- Analytics dashboard with campaign performance
- Asset library management

Success Metrics:

- Brand admins can create campaigns in under 5 minutes
- Store selection supports 10,000+ stores
- Analytics load within 2 seconds
- Asset library handles files up to 100MB

S4: PSP Operations Portal (Week 9-10)

Goal: Enable PSP teams to oversee campaign execution and logistics.

Key Objectives:

- PSP dashboard with operational metrics
- Campaign oversight with multi-brand view
- Shipment tracking and management
- Print queue management
- Reporting and export capabilities

Success Metrics:

- PSP operators can monitor all active campaigns
- Shipment status updates in real-time
- Print queue processes 1000+ items
- Reports export in under 30 seconds

S5: Store Portal & Integration (Week 11-12)

Goal: Complete the store-facing experience and integrate all system components.

Key Objectives:

- Store portal with location-specific content
- Campaign calendar with upcoming installations
- Display compliance tracking and scoring
- Integration with external systems (ERP, shipping)

- Notification center with preferences

Success Metrics:

- Store managers access portal within 3 seconds
- Calendar syncs with campaign deadlines
- Compliance scores calculate accurately
- External integrations pass validation tests

S6: Polish, Testing & Beta Launch (Week 13)

Goal: Ensure production readiness and launch beta program.

Key Objectives:

- End-to-end testing across all portals
- Performance optimization and caching
- Security audit and penetration testing
- Documentation completion
- Beta program launch with select partners

Success Metrics:

- All critical paths pass E2E tests
- Page load times under 2 seconds
- Zero critical security vulnerabilities
- 95% uptime during beta period

Resource Allocation

Role	S0	S1	S2	S3	S4	S5	S6
Frontend	1	2	2	2	2	2	1
Backend	2	1	1	2	2	2	1
Mobile	0	1	2	1	0	1	1
DevOps	2	1	0	0	0	1	1
QA	0	1	1	1	1	1	2

Risk Mitigation

Risk	Probability	Impact	Mitigation

Scope creep	High	High	Strict sprint boundaries, change control process
Technical debt	Medium	Medium	20% buffer in each sprint for refactoring
Resource constraints	Medium	High	Cross-training, documentation
Integration delays	Medium	Medium	Mock services, early integration testing
Performance issues	Low	High	Performance budgets, continuous monitoring

Communication Cadence

- **Daily:** Stand-up meetings (15 minutes)
- **Weekly:** Sprint progress review
- **Bi-weekly:** Sprint retrospective and planning
- **Monthly:** Stakeholder demo and feedback session

Related Documents

- Sprint Calendar - Date-bound schedule
- Dependency Map - Cross-sprint dependencies
- Sprint Details - Individual sprint documentation

Last Updated: 2026-01-01

Sprint S0: Foundation & Infrastructure

Sprint Overview

Attribute	Value
Sprint Number	S0
Theme	Foundation & Infrastructure
Start Date	2026-01-06
End Date	2026-01-17
Duration	2 weeks (10 working days)
Sprint Goal	Establish the technical foundation that all subsequent development will build upon

Sprint Goals

1. Set up Turborepo monorepo structure with shared packages
2. Implement authentication foundation with Supabase
3. Configure PostgreSQL database with initial schema
4. Establish CI/CD pipelines with GitHub Actions
5. Configure development, staging, and production environments

---

Task Breakdown

Task ID	Description	Priority	Estimate	Assignee	Status
S0-01	Initialize Turborepo monorepo structure	Critical	8h	TBD	Not Started
S0-02	Configure shared TypeScript configuration	Critical	4h	TBD	Not Started
S0-03	Set up shared ESLint and Prettier configs	High	4h	TBD	Not Started
S0-04	Create shared UI component package structure	High	8h	TBD	Not Started
S0-05	Initialize Next.js app for Brand Portal	High	4h	TBD	Not Started

S0-06	Initialize Next.js app for PSP Portal	High	4h	TBD	Not Started
S0-07	Initialize Next.js app for Store Portal	High	4h	TBD	Not Started
S0-08	Initialize Next.js PWA for Field App	High	8h	TBD	Not Started
S0-09	Initialize Fastify API Server	Critical	4h	TBD	Not Started
S0-10	Implement JWT Auth in Fastify	Critical	12h	TBD	Not Started
S0-11	Design and implement Drizzle/Postgres initial schema	Critical	16h	TBD	Not Started
S0-12	Set up database migrations with Drizzle Kit	High	8h	TBD	Not Started
S0-13	Configure API Route Validation (Zod)	High	8h	TBD	Not Started
S0-14	Create seed data scripts	Medium	4h	TBD	Not Started
S0-15	Set up GitHub Actions CI pipeline	Critical	8h	TBD	Not Started
S0-16	Configure automated testing in CI	High	8h	TBD	Not Started
S0-17	Set up Vercel deployment for web apps	High	4h	TBD	Not Started
S0-18	Configure Docker Build for backend	High	8h	TBD	Not Started
S0-19	Set up development environment variables	High	4h	TBD	Not Started
S0-20	Configure staging environment	High	4h	TBD	Not Started
S0-21	Configure production environment	Medium	4h	TBD	Not Started
S0-22	Document local development setup	High	4h	TBD	Not Started
S0-23	Create architecture decision records (ADRs)	Medium	4h	TBD	Not Started
S0-24	Set up error monitoring (Sentry)	Medium	4h	TBD	Not Started

S0-25	Configure logging infrastructure	Medium	4h	TBD	Not Started

---

Task Details

S0-01: Initialize Turborepo Monorepo Structure

Description: Create the foundational Turborepo monorepo structure with proper workspace configuration.

Technical Details:

- Initialize Turborepo with pnpm workspaces
- Configure turbo.json with pipeline definitions
- Set up apps/ directory for web and mobile applications
- Set up packages/ directory for shared code
- Configure root package.json with workspace scripts

Acceptance Criteria:

- [ ] Turborepo initialized with proper configuration
- [ ] pnpm workspaces configured correctly
- [ ] Pipeline runs build, lint, and test across all packages
- [ ] Hot module reload works in development

S0-09: Initialize Fastify API Server

Description: Initialize the Fastify backend service with proper plugin architecture.

Technical Details:

- Set up Fastify instance with TypeScript
- Configure standard plugins (cors, helmet, swagger)
- Set up Zod type provider
- Configure error handling and logging (pino)
- Establish Docker health check endpoints

Acceptance Criteria:

- [ ] Fastify server runs successfully
- [ ] Swagger documentation accessible at /documentation
- [ ] Zod validation working for simple route
- [ ] Docker container builds successfully

S0-11: Design and Implement Drizzle/Postgres Initial Schema

Description: Create the initial database schema using Drizzle ORM code-first approach.

Technical Details:

- Users and Authentication tables (refresh tokens)
- Organizations (Tenants)
- Stores and Locations
- Campaigns and Assignments
- JSONB columns for flexible configuration

Acceptance Criteria:

- [ ] Schema defined in `src/db/schema.ts`
- [ ] Relations defined using Drizzle relations API
- [ ] Migrations generated via `drizzle-kit generate`
- [ ] Migrations applied successfully to local Postgres
- [ ] ERD matches architectural requirements

S0-15: Set Up GitHub Actions CI Pipeline

Description: Implement continuous integration pipeline for automated testing and builds.

Technical Details:

- Trigger on push and pull request
- Matrix testing across Node versions
- Parallel execution of lint, type-check, and test
- Cache dependencies for faster builds
- Status checks required for merge

Acceptance Criteria:

- [ ] Pipeline triggers on all PRs
- [ ] All quality checks execute in parallel
- [ ] Build artifacts cached properly
- [ ] Average pipeline duration under 5 minutes
- [ ] Failed checks block PR merge

Acceptance Criteria

Sprint Completion Criteria

- [ ] All team members can clone repo and run locally in under 15 minutes
- [ ] CI pipeline passes on all commits to main branch

- [ ] Database migrations run successfully in all environments
- [ ] Authentication flow works end-to-end (signup, login, logout)
- [ ] All three web apps and mobile app build successfully
- [ ] Environment variables documented and secured
- [ ] Local development guide completed and tested

Definition of Done

- [ ] Code reviewed and approved by at least one team member
- [ ] Unit tests written for new functionality
- [ ] No linting errors or warnings
- [ ] TypeScript strict mode passing
- [ ] Documentation updated
- [ ] Deployed to staging environment

Dependencies

External Dependencies

Dependency	Source	Status	Risk
Docker Hub	Docker	Available	Low
Vercel Account	Vercel	Available	Low
GitHub Repository	GitHub	Available	Low
Apple Developer	Apple	Available	Low

Incoming Dependencies

None - S0 is the foundation sprint.

Outgoing Dependencies

Blocked Task	Blocked Sprint	Impact
S1-01 Universal Login	S1	Critical - Auth required
S1-03 Core Layouts	S1	Critical - Monorepo required
S1-04 Role-based Routing	S1	High - Database required

Risks and Mitigations

Risk	Probability	Impact	Mitigation
Fastify plugin complexity	Low	Medium	Use established community plugins
Turborepo learning curve	Low	Medium	Team training session planned
Database schema changes	Medium	Medium	Design review with stakeholders
CI pipeline debugging	Medium	Low	Incremental pipeline development

---

Team Allocation

Role	Team Member	Capacity	Focus Areas
Tech Lead	TBD	100%	Architecture, reviews
Backend Dev 1	TBD	100%	Database, Fastify API
Backend Dev 2	TBD	100%	CI/CD, environments
Frontend Dev	TBD	50%	Monorepo, app setup
DevOps	TBD	100%	Infrastructure, deployments

Sprint Ceremonies

Ceremony	Date	Time	Duration
Sprint Planning	Jan 6, 2026	10:00 AM	2 hours
Daily Standup	Daily	9:00 AM	15 minutes
Mid-Sprint Review	Jan 10, 2026	2:00 PM	1 hour
Sprint Review	Jan 17, 2026	2:00 PM	1 hour
Sprint Retrospective	Jan 17, 2026	3:30 PM	1 hour

Success Metrics

Metric	Target	Measurement
Setup time for new developer	< 15 minutes	Timed test
CI pipeline duration	< 5 minutes	GitHub Actions

Build success rate	100%	CI metrics
Code coverage baseline	> 50%	Jest coverage

Notes

- Week 1 focus: Monorepo structure, API setup, database schema
- Week 2 focus: CI/CD, environments, documentation
- Daily syncs at 9 AM to unblock issues quickly
- Architecture decisions documented in ADRs

Related Documents

- Sprint Roadmap
- Sprint Calendar
- Dependency Map

Last Updated: 2026-01-01

Sprint S1: Authentication & Core UI

Sprint Overview

Attribute	Value
Sprint Number	S1
Theme	Authentication & Core UI
Start Date	2026-01-20
End Date	2026-01-31
Duration	2 weeks (10 working days)
Sprint Goal	Deliver a polished, secure authentication experience and establish core UI patterns

Sprint Goals

1. Implement universal login page with role-based redirects
2. Complete multi-factor authentication (MFA) implementation
3. Build core layout components (headers, navigation, sidebars)
4. Establish role-based routing and access control
5. Implement design system with Tailwind CSS and document in Storybook

---

Task Breakdown

Task ID	Description	Priority	Estimate	Assignee	Status
S1-01	Create universal login page component	Critical	8h	TBD	Not Started
S1-02	Implement email/password authentication flow	Critical	8h	TBD	Not Started
S1-03	Add social login providers (Google, Microsoft)	High	8h	TBD	Not Started
S1-04	Implement password reset flow	High	6h	TBD	Not Started
S1-05	Create role-based redirect logic	Critical	8h	TBD	Not Started

S1-06	Implement MFA enrollment UI	High	8h	TBD	Not Started
S1-07	Integrate TOTP authenticator support	High	8h	TBD	Not Started
S1-08	Add SMS-based MFA option	Medium	6h	TBD	Not Started
S1-09	Create MFA challenge/verification flow	High	8h	TBD	Not Started
S1-10	Build recovery codes generation and validation	Medium	6h	TBD	Not Started
S1-11	Design and implement header component	Critical	8h	TBD	Not Started
S1-12	Create sidebar navigation component	Critical	8h	TBD	Not Started
S1-13	Build responsive layout wrapper	High	6h	TBD	Not Started
S1-14	Implement breadcrumb navigation	Medium	4h	TBD	Not Started
S1-15	Create page container components	High	4h	TBD	Not Started
S1-16	Define user roles and permissions matrix	Critical	4h	TBD	Not Started
S1-17	Implement route guards for protected pages	Critical	8h	TBD	Not Started
S1-18	Create permission-based component rendering	High	6h	TBD	Not Started
S1-19	Build role-specific navigation menus	High	6h	TBD	Not Started
S1-20	Implement session management and timeout	High	6h	TBD	Not Started
S1-21	Configure Tailwind CSS theme and tokens	Critical	8h	TBD	Not Started
S1-22	Create button component variants	High	4h	TBD	Not Started
S1-23	Build form input components	High	6h	TBD	Not Started
S1-24	Create modal and dialog components	High	6h	TBD	Not Started

S1-25	Build card and panel components	High	4h	TBD	Not Started
S1-26	Create data table component	High	8h	TBD	Not Started
S1-27	Set up Storybook with all components	High	8h	TBD	Not Started
S1-28	Document component API and usage	Medium	6h	TBD	Not Started
S1-29	Implement accessibility (WCAG 2.1 AA)	High	8h	TBD	Not Started
S1-30	Add loading states and skeleton screens	Medium	6h	TBD	Not Started

---

Task Details

S1-01: Create Universal Login Page Component

Description: Build the main login page that serves as the entry point for all users.

Technical Details:

- Responsive design for mobile and desktop
- Clean, professional UI matching brand guidelines
- Form validation with real-time feedback
- Support for email and social login options
- Error handling with user-friendly messages

Acceptance Criteria:

- [ ] Login form displays correctly on all screen sizes
- [ ] Form validation prevents submission of invalid data
- [ ] Error messages are clear and actionable
- [ ] Loading state shown during authentication
- [ ] Successful login redirects to appropriate portal

S1-05: Create Role-Based Redirect Logic

Description: Implement logic to redirect users to the correct portal based on their role.

Technical Details:

- Fetch user role from API /decrypted JWT token

- Map roles to portal URLs (Brand -> /brand, PSP -> /psp, Store -> /store)
- Handle users with multiple roles (role selector)
- Implement default landing pages per role
- Support deep linking with role verification

Acceptance Criteria:

- [ ] Store associates redirected to mobile app or store portal
- [ ] Brand admins redirected to brand portal
- [ ] PSP operators redirected to PSP portal
- [ ] Multi-role users shown role selector
- [ ] Unauthorized access attempts blocked and logged

S1-11: Design and Implement Header Component

Description: Create the main header component used across all web portals.

Technical Details:

- Logo with link to dashboard
- User menu with profile, settings, logout
- Notification bell with unread count
- Search bar (if applicable to portal)
- Mobile hamburger menu trigger
- Role/organization indicator

Acceptance Criteria:

- [ ] Header displays correctly on desktop and mobile
- [ ] User menu opens/closes properly
- [ ] Notification count updates in real-time
- [ ] Mobile menu accessible and functional
- [ ] Consistent across all portals with role-specific customization

---

S1-21: Configure Tailwind CSS Theme and Tokens

Description: Set up the design token system using Tailwind CSS configuration.

Technical Details:

- Define color palette (primary, secondary, semantic colors)
- Configure typography scale
- Set up spacing and sizing tokens
- Define shadow and border radius scales
- Configure responsive breakpoints
- Add custom utility classes

Acceptance Criteria:

- [ ] All design tokens defined in Tailwind config
- [ ] Colors include light and dark variants
- [ ] Typography scale matches design specifications
- [ ] Tokens documented with usage examples
- [ ] Theme preview available in Storybook

Acceptance Criteria

Sprint Completion Criteria

- [ ] Users can log in with email/password
- [ ] Users can log in with Google and Microsoft
- [ ] MFA can be enrolled and verified
- [ ] Users are redirected to correct portal based on role
- [ ] All core layout components implemented and documented
- [ ] Design system components available in Storybook
- [ ] 100% accessibility compliance (WCAG 2.1 AA)
- [ ] All pages responsive from 320px to 1920px

Definition of Done

- [ ] Code reviewed and approved by at least one team member
- [ ] Unit tests written with >80% coverage
- [ ] Accessibility tests passing
- [ ] Visual regression tests passing
- [ ] Components documented in Storybook
- [ ] No linting errors or warnings
- [ ] Deployed to staging environment

Dependencies

Incoming Dependencies

From Sprint	Task	Required By	Status
From Sprint	Task	Required By	Status
----------	------	----------	--------
S0	S0-01 Monorepo Setup	S1-01	TBD
S0	S0-10 JWT Auth	S1-01, S1-02	TBD

S0	S0-11 Drizzle Schema	S1-16	TBD
S0	S0-15 CI/CD Pipeline	S1-27	TBD

Outgoing Dependencies

Blocked Task	Blocked Sprint	Impact
S2-01 Store Dashboard	S2	Critical - Layouts required
S3-01 Brand Dashboard	S3	Critical - Layouts required
S4-01 PSP Dashboard	S4	Critical - Layouts required
S5-01 Store Portal	S5	Critical - Layouts required

Risks and Mitigations

Risk	Probability	Impact	Mitigation
Social login provider issues	Medium	Medium	Test early, have fallback
MFA complexity	Medium	High	Phased rollout, thorough testing
Accessibility compliance	Low	High	Early testing, automated checks
Design system scope creep	Medium	Medium	Strict MVP component list

---

Team Allocation

Role	Team Member	Capacity	Focus Areas
Tech Lead	TBD	50%	Architecture, reviews
Frontend Dev 1	TBD	100%	Login, MFA flows
Frontend Dev 2	TBD	100%	Layouts, design system
Backend Dev	TBD	50%	Auth APIs, permissions
Mobile Dev	TBD	50%	Mobile auth integration
QA Engineer	TBD	50%	Testing, accessibility

Sprint Ceremonies

Ceremony	Date	Time	Duration
Sprint Planning	Jan 20, 2026	10:00 AM	2 hours
Daily Standup	Daily	9:00 AM	15 minutes
Mid-Sprint Review	Jan 24, 2026	2:00 PM	1 hour
Sprint Review	Jan 31, 2026	2:00 PM	1 hour
Sprint Retrospective	Jan 31, 2026	3:30 PM	1 hour

Success Metrics

Metric	Target	Measurement
Login success rate	> 95%	Analytics
Time to complete login	< 10 seconds	Performance monitoring
Accessibility score	100%	axe DevTools
Component coverage in Storybook	100%	Manual review
Unit test coverage	> 80%	Jest coverage

Notes

- Week 3 focus: Login flows, MFA, core layouts
- Week 4 focus: Role routing, design system, Storybook
- Accessibility testing daily with axe DevTools
- Design review mid-sprint to catch issues early
- MLK Day (Jan 20) - reduced capacity, plan accordingly

Related Documents

- Sprint Roadmap
- Sprint Calendar
- Dependency Map
- Sprint S0: Foundation

Last Updated: 2026-01-01

Sprint S2: Mobile App Core

Sprint Overview

Attribute	Value
Sprint Number	S2
Theme	Mobile App Core
Start Date	2026-02-03
End Date	2026-02-14
Duration	2 weeks (10 working days)
Sprint Goal	Build the essential mobile app functionality for store associates

Sprint Goals

1. Create store dashboard with campaign overview
2. Implement campaign list with status indicators
3. Build photo capture with AI-powered quality validation
4. Develop checklist execution with offline support
5. Integrate push notification system

Task Breakdown

Task ID	Description	Priority	Estimate	Assignee	Status
S2-01	Create store associate dashboard screen	Critical	8h	TBD	Not Started
S2-02	Implement campaign summary cards	High	6h	TBD	Not Started
S2-03	Build quick action buttons (scan, photo, checklist)	High	6h	TBD	Not Started
S2-04	Create notification banner component	Medium	4h	TBD	Not Started
S2-05	Implement bottom navigation bar	Critical	6h	TBD	Not Started

S2-06	Build campaign list screen	Critical	8h	TBD	Not Started
S2-07	Create campaign card component with status	High	6h	TBD	Not Started
S2-08	Implement campaign filtering (status, date)	High	6h	TBD	Not Started
S2-09	Build campaign detail screen	High	8h	TBD	Not Started
S2-10	Create campaign timeline/progress view	Medium	6h	TBD	Not Started
S2-11	Implement photo capture screen	Critical	8h	TBD	Not Started
S2-12	Build camera controls (flash, switch, zoom)	High	6h	TBD	Not Started
S2-13	Create photo preview and retake flow	High	6h	TBD	Not Started
S2-14	Implement AI quality validation service	Critical	16h	TBD	Not Started
S2-15	Build quality feedback UI (blur, lighting, framing)	High	8h	TBD	Not Started
S2-16	Create photo gallery for campaign submissions	Medium	6h	TBD	Not Started
S2-17	Build checklist execution screen	Critical	8h	TBD	Not Started
S2-18	Implement checklist item components (checkbox, photo, text)	High	8h	TBD	Not Started
S2-19	Create progress indicator for checklists	Medium	4h	TBD	Not Started
S2-20	Build offline data persistence (AsyncStorage/WatermelonDB)	Critical	12h	TBD	Not Started
S2-21	Implement background sync service	Critical	12h	TBD	Not Started
S2-22	Create sync status indicator	High	4h	TBD	Not Started
S2-23	Set up Firebase Cloud Messaging	Critical	8h	TBD	Not Started

S2-24	Implement push notification handlers	High	6h	TBD	Not Started
S2-25	Create in-app notification center	Medium	6h	TBD	Not Started
S2-26	Build notification preferences screen	Medium	4h	TBD	Not Started
S2-27	Implement deep linking for notifications	High	6h	TBD	Not Started
S2-28	Add error handling and retry logic	High	8h	TBD	Not Started
S2-29	Create loading and empty states	Medium	4h	TBD	Not Started
S2-30	Implement haptic feedback for interactions	Low	2h	TBD	Not Started

---

Task Details

S2-01: Create Store Associate Dashboard Screen

Description: Build the main dashboard that store associates see upon login.

Technical Details:

- Today's tasks summary with counts
- Active campaign cards (max 3, with "view all" link)
- Quick action buttons for common tasks
- Greeting with user name and store location
- Pull-to-refresh functionality
- Real-time updates via WebSocket

Acceptance Criteria:

- [ ] Dashboard loads within 2 seconds
- [ ] Campaign summaries show accurate counts
- [ ] Quick actions navigate to correct screens
- [ ] Pull-to-refresh updates data
- [ ] Offline mode shows cached data with indicator

S2-11: Implement Photo Capture Screen

Description: Create the camera interface for capturing display verification photos.

Technical Details:

- Full-screen camera preview
- Capture button with visual feedback
- Flash toggle (auto, on, off)
- Camera switch (front/back)
- Grid overlay option for composition
- Zoom controls (pinch and buttons)
- Resolution optimization for upload

Acceptance Criteria:

- [ ] Camera launches within 1 second
- [ ] Capture provides haptic and visual feedback
- [ ] Photos captured at appropriate resolution (2MP-5MP)
- [ ] Flash controls work correctly
- [ ] Camera permissions handled gracefully

S2-14: Implement AI Quality Validation Service

Description: Build the AI-powered service that validates photo quality before submission.

Technical Details:

- Integration with Cloud Vision or custom ML model
- Check for blur detection
- Verify adequate lighting
- Confirm display is fully in frame
- Detect obstructions or reflections
- Provide actionable feedback messages
- Allow override with confirmation

Acceptance Criteria:

- [ ] Validation completes within 3 seconds
- [ ] Blur detection accuracy > 90%
- [ ] Lighting assessment provides clear guidance
- [ ] Framing detection identifies cut-off displays
- [ ] Users can override with acknowledgment

S2-20: Build Offline Data Persistence

Description: Implement offline-first data architecture for checklist and photo data.

Technical Details:

- WatermelonDB for structured data
- AsyncStorage for user preferences
- Image caching for offline access
- Conflict resolution strategy
- Data encryption at rest
- Storage quota management

Acceptance Criteria:

- [ ] Checklists accessible without network
- [ ] Photos cached locally before upload
- [ ] Data persists across app restarts
- [ ] Sync conflicts resolved correctly
- [ ] Storage usage under 500MB

Acceptance Criteria

Sprint Completion Criteria

- [ ] Store associates can view dashboard with campaigns
- [ ] Campaign list shows all assigned campaigns with status
- [ ] Photos can be captured with quality validation
- [ ] Checklists can be completed offline
- [ ] Data syncs when connectivity restored
- [ ] Push notifications received and displayed
- [ ] App works offline for core functionality

Definition of Done

- [ ] Code reviewed and approved
- [ ] Unit tests with >70% coverage
- [ ] Integration tests for critical paths
- [ ] Manual testing on iOS and Android
- [ ] Performance profiling completed
- [ ] No memory leaks detected
- [ ] Deployed to TestFlight and Play Store Internal

Dependencies

Incoming Dependencies

From Sprint	Task	Required By	Status

S1	S1-01 Universal Login	S2-01	TBD
S1	S1-03 Core Layouts	S2-01	TBD
S1	S1-04 Role-based Routing	S2-06	TBD
S1	S1-05 Design System	S2-01	TBD

Outgoing Dependencies

Blocked Task	Blocked Sprint	Impact
S3-02 Campaign Management	S3	High - Campaign data
S3-04 Analytics Dashboard	S3	Medium - Photo data
S5-03 Display Compliance	S5	High - Checklist data

Risks and Mitigations

Risk	Probability	Impact	Mitigation
Camera API differences iOS/Android	Medium	High	Early testing on both platforms
AI validation accuracy	Medium	Medium	Training data review, fallback
Offline sync complexity	High	High	Thorough edge case testing
Push notification delivery	Medium	Medium	Fallback to in-app polling
Performance on older devices	Medium	Medium	Performance budgets, testing

---

Team Allocation

Role	Team Member	Capacity	Focus Areas
Tech Lead	TBD	25%	Architecture, reviews
Mobile Dev 1	TBD	100%	Dashboard, campaigns
Mobile Dev 2	TBD	100%	Photo capture, AI validation
Backend Dev	TBD	50%	APIs, push notifications
ML Engineer	TBD	50%	AI quality validation

QA Engineer	TBD	50%	Mobile testing

Sprint Ceremonies

Ceremony	Date	Time	Duration
Sprint Planning	Feb 3, 2026	10:00 AM	2 hours
Daily Standup	Daily	9:00 AM	15 minutes
Mid-Sprint Review	Feb 7, 2026	2:00 PM	1 hour
Sprint Review	Feb 14, 2026	2:00 PM	1 hour
Sprint Retrospective	Feb 14, 2026	3:30 PM	1 hour

Success Metrics

Metric	Target	Measurement
App launch time	< 3 seconds	Performance monitoring
Photo capture to validation	< 5 seconds	In-app timing
Offline mode reliability	99%	Error tracking
Push notification delivery	> 95%	FCM analytics
Crash-free rate	> 99%	Crashlytics

Notes

- Week 5 focus: Dashboard, campaign list, navigation
- Week 6 focus: Photo capture, checklists, offline sync
- Daily testing on physical devices required
- AI validation model training in parallel
- Consider React Native performance optimizations

Related Documents

- Sprint Roadmap
- Sprint Calendar
- Dependency Map
- Sprint S1: Auth & Core UI

Last Updated: 2026-01-01

Sprint S3: Brand Admin Portal

Sprint Overview

Attribute	Value
Sprint Number	S3
Theme	Brand Admin Portal
Start Date	2026-02-17
End Date	2026-02-28
Duration	2 weeks (10 working days)
Sprint Goal	Empower brand administrators to manage campaigns effectively

Sprint Goals

1. Build brand dashboard with key performance metrics
2. Implement campaign creation and management workflow
3. Create store selection with filtering and bulk actions
4. Develop analytics dashboard with campaign performance
5. Build asset library management system

Task Breakdown

Task ID	Description	Priority	Estimate	Assignee	Status
S3-01	Create brand admin dashboard layout	Critical	8h	TBD	Not Started
S3-02	Build KPI cards (active campaigns, stores, completion rate)	High	6h	TBD	Not Started
S3-03	Implement campaign performance chart	High	8h	TBD	Not Started
S3-04	Create recent activity feed	Medium	6h	TBD	Not Started
S3-05	Build quick action shortcuts	Medium	4h	TBD	Not Started

S3-06	Create campaign wizard - step 1 (basic info)	Critical	8h	TBD	Not Started
S3-07	Build campaign wizard - step 2 (targeting)	Critical	8h	TBD	Not Started
S3-08	Implement campaign wizard - step 3 (assets)	Critical	8h	TBD	Not Started
S3-09	Create campaign wizard - step 4 (schedule)	High	6h	TBD	Not Started
S3-10	Build campaign wizard - step 5 (review & submit)	High	6h	TBD	Not Started
S3-11	Implement campaign list with status filters	High	8h	TBD	Not Started
S3-12	Create campaign detail/edit page	High	8h	TBD	Not Started
S3-13	Build campaign duplication feature	Medium	4h	TBD	Not Started
S3-14	Implement campaign archiving	Medium	4h	TBD	Not Started
S3-15	Create store selection interface	Critical	12h	TBD	Not Started
S3-16	Build store filter panel (region, type, tags)	High	8h	TBD	Not Started
S3-17	Implement bulk store selection	High	6h	TBD	Not Started
S3-18	Create store group management	Medium	6h	TBD	Not Started
S3-19	Build store map visualization	Medium	8h	TBD	Not Started
S3-20	Create analytics dashboard layout	Critical	8h	TBD	Not Started
S3-21	Implement campaign completion metrics	High	8h	TBD	Not Started
S3-22	Build store performance leaderboard	Medium	6h	TBD	Not Started
S3-23	Create photo submission gallery	High	8h	TBD	Not Started
S3-24	Implement data export functionality	Medium	6h	TBD	Not Started

S3-25	Build asset library upload interface	High	8h	TBD	Not Started
S3-26	Create asset organization (folders, tags)	High	6h	TBD	Not Started
S3-27	Implement asset preview and download	Medium	4h	TBD	Not Started
S3-28	Build asset usage tracking	Low	4h	TBD	Not Started
S3-29	Create brand settings page	Medium	6h	TBD	Not Started
S3-30	Implement team member management	Medium	6h	TBD	Not Started

---

Task Details

S3-01: Create Brand Admin Dashboard Layout

Description: Build the main dashboard interface for brand administrators.

Technical Details:

- Responsive grid layout for metrics
- Chart.js or Recharts for visualizations
- Real-time data updates with SWR/React Query
- Date range selector for metrics
- Export dashboard as PDF option

Acceptance Criteria:

- [ ] Dashboard loads within 2 seconds
- [ ] All KPI cards show accurate data
- [ ] Charts are interactive with tooltips
- [ ] Date range filtering works correctly
- [ ] Responsive design for tablet and desktop

S3-06: Create Campaign Wizard - Step 1 (Basic Info)

Description: Build the first step of the campaign creation wizard.

Technical Details:

- Campaign name with uniqueness validation
- Campaign description (rich text editor)

- Campaign type selection
- Brand product selection
- Campaign goals setting
- Draft auto-save functionality

Acceptance Criteria:

- [ ] Form validates required fields
- [ ] Auto-save every 30 seconds
- [ ] Character counts displayed
- [ ] Product selection from dropdown
- [ ] Can navigate to next step only when valid

S3-15: Create Store Selection Interface

Description: Build the interface for selecting target stores for campaigns.

Technical Details:

- Virtual scrolling for 10,000+ stores
- Multi-select with keyboard support
- Filter by region, state, city, type
- Search by store name or ID
- Saved selection templates
- Selection count indicator

Acceptance Criteria:

- [ ] Interface handles 10,000+ stores smoothly
- [ ] Filters apply in under 500ms
- [ ] Bulk select all filtered results
- [ ] Selection persists across filter changes
- [ ] Can save selection as template

S3-20: Create Analytics Dashboard Layout

Description: Build the analytics dashboard for campaign performance insights.

Technical Details:

- Campaign selector dropdown
- Date range comparison
- Completion rate over time chart
- Store performance breakdown
- Geographic heatmap
- Export to Excel/CSV

Acceptance Criteria:

- [ ] Dashboard loads within 3 seconds
- [ ] Date comparison shows delta indicators
- [ ] Charts update on filter change
- [ ] Export includes all visible data
- [ ] Drill-down to individual stores

Acceptance Criteria

Sprint Completion Criteria

- [ ] Brand admins can view dashboard with KPIs
- [ ] Campaign creation wizard completes end-to-end
- [ ] Store selection supports 10,000+ stores
- [ ] Analytics dashboard shows campaign performance
- [ ] Asset library allows upload and organization
- [ ] All pages responsive and accessible

Definition of Done

- [ ] Code reviewed and approved
- [ ] Unit tests with >80% coverage
- [ ] E2E tests for campaign creation flow
- [ ] Performance testing completed
- [ ] Accessibility audit passed
- [ ] Deployed to staging environment

Dependencies

Incoming Dependencies

From Sprint	Task	Required By	Status
S1	S1-01 Universal Login	S3-01	TBD
S1	S1-03 Core Layouts	S3-01	TBD
S1	S1-04 Role-based Routing	S3-06	TBD
S2	S2-06 Campaign List	S3-11	TBD
S2	S2-11 Photo Capture	S3-23	TBD

Outgoing Dependencies

Blocked Task	Blocked Sprint	Impact
S4-02 Campaign Oversight	S4	Critical - Campaign data
S4-01 PSP Dashboard	S4	High - Store data
S4-05 Reporting	S4	High - Analytics data

Risks and Mitigations

Risk	Probability	Impact	Mitigation
Store selection performance	Medium	High	Virtual scrolling, pagination
Campaign wizard complexity	Medium	Medium	Modular step components
Analytics data volume	Medium	High	Aggregation, caching
Asset upload large files	Low	Medium	Chunked upload, progress

Team Allocation

Role	Team Member	Capacity	Focus Areas
Tech Lead	TBD	25%	Architecture, reviews
Frontend Dev 1	TBD	100%	Dashboard, analytics
Frontend Dev 2	TBD	100%	Campaign wizard, store selection
Backend Dev 1	TBD	100%	Campaign APIs, analytics
Backend Dev 2	TBD	50%	Asset management APIs
QA Engineer	TBD	50%	Testing, E2E

Sprint Ceremonies

Ceremony	Date	Time	Duration
Sprint Planning	Feb 17, 2026	10:00 AM	2 hours
Daily Standup	Daily	9:00 AM	15 minutes
Mid-Sprint Review	Feb 21, 2026	2:00 PM	1 hour
Sprint Review	Feb 28, 2026	2:00 PM	1 hour

Sprint Retrospective	Feb 28, 2026	3:30 PM	1 hour

Success Metrics

Metric	Target	Measurement
Campaign creation time	< 5 minutes	User testing
Store selection performance	< 500ms filter	Performance monitoring
Analytics load time	< 3 seconds	Performance monitoring
Asset upload speed	10MB in < 30s	Manual testing
User satisfaction	> 4/5 rating	User feedback

Notes

- Week 7 focus: Dashboard, campaign wizard (steps 1-3)
- Week 8 focus: Store selection, analytics, asset library
- Presidents Day (Feb 16) - day before sprint, plan accordingly
- Consider lazy loading for analytics charts
- Asset library should support drag-and-drop

Related Documents

- Sprint Roadmap
- Sprint Calendar
- Dependency Map
- Sprint S2: Mobile Core

Last Updated: 2026-01-01

Sprint S4: PSP Operations Portal

Sprint Overview

Attribute	Value
Sprint Number	S4
Theme	PSP Operations Portal
Start Date	2026-03-03
End Date	2026-03-14
Duration	2 weeks (10 working days)
Sprint Goal	Enable PSP teams to oversee campaign execution and logistics

Sprint Goals

1. Build PSP dashboard with operational metrics
2. Implement campaign oversight with multi-brand view
3. Create shipment tracking and management system
4. Develop print queue management interface
5. Build reporting and export capabilities

Task Breakdown

Task ID	Description	Priority	Estimate	Assignee	Status
S4-01	Create PSP operations dashboard layout	Critical	8h	TBD	Not Started
S4-02	Build operational KPI cards	High	6h	TBD	Not Started
S4-03	Implement campaign volume chart	High	6h	TBD	Not Started
S4-04	Create production schedule calendar	High	8h	TBD	Not Started
S4-05	Build alert notification panel	Medium	6h	TBD	Not Started

S4-06	Create multi-brand campaign list	Critical	8h	TBD	Not Started
S4-07	Build brand filter and grouping	High	6h	TBD	Not Started
S4-08	Implement campaign status overview	High	6h	TBD	Not Started
S4-09	Create campaign drill-down view	High	8h	TBD	Not Started
S4-10	Build store execution tracking	High	8h	TBD	Not Started
S4-11	Implement bulk status updates	Medium	6h	TBD	Not Started
S4-12	Create shipment list view	Critical	8h	TBD	Not Started
S4-13	Build shipment detail panel	High	6h	TBD	Not Started
S4-14	Implement carrier integration (tracking)	Critical	12h	TBD	Not Started
S4-15	Create shipment creation workflow	High	8h	TBD	Not Started
S4-16	Build batch shipment processing	Medium	8h	TBD	Not Started
S4-17	Implement delivery confirmation flow	High	6h	TBD	Not Started
S4-18	Create print queue dashboard	Critical	8h	TBD	Not Started
S4-19	Build job list with filtering	High	6h	TBD	Not Started
S4-20	Implement priority management	High	6h	TBD	Not Started
S4-21	Create job status workflow	High	8h	TBD	Not Started
S4-22	Build print specification viewer	Medium	6h	TBD	Not Started
S4-23	Implement batch printing actions	Medium	6h	TBD	Not Started
S4-24	Create report builder interface	High	12h	TBD	Not Started

S4-25	Build pre-defined report templates	High	8h	TBD	Not Started
S4-26	Implement scheduled report delivery	Medium	6h	TBD	Not Started
S4-27	Create export to Excel/CSV/PDF	High	8h	TBD	Not Started
S4-28	Build report history and favorites	Low	4h	TBD	Not Started
S4-29	Create PSP settings page	Medium	4h	TBD	Not Started
S4-30	Implement user management for PSP	Medium	6h	TBD	Not Started

---

Task Details

S4-01: Create PSP Operations Dashboard Layout

Description: Build the main dashboard for PSP operations managers.

Technical Details:

- Multi-brand summary view
- Active campaigns count by brand
- Production queue status
- Shipment status overview
- Upcoming deadlines calendar widget
- Real-time refresh (30-second interval)

Acceptance Criteria:

- [ ] Dashboard shows all brands with active campaigns
- [ ] KPI cards update in real-time
- [ ] Calendar shows next 7 days of deadlines
- [ ] Alert panel shows critical items
- [ ] Drill-down links work correctly

S4-06: Create Multi-Brand Campaign List

Description: Build the campaign oversight list that shows all brands' campaigns.

Technical Details:

- Grouped by brand with expand/collapse

- Status indicators (on track, at risk, delayed)
- Sortable columns
- Advanced filtering
- Bulk action toolbar
- Export visible results

Acceptance Criteria:

- [ ] All brands' campaigns visible
- [ ] Grouping toggles work correctly
- [ ] Status calculations accurate
- [ ] Filtering applies instantly
- [ ] Bulk actions process correctly

S4-14: Implement Carrier Integration (Tracking)

Description: Integrate with shipping carriers for real-time tracking updates.

Technical Details:

- Integration with UPS, FedEx, USPS APIs
- Webhook handlers for tracking updates
- Status normalization across carriers
- Delivery exception detection
- Estimated delivery date calculation
- Proof of delivery capture

Acceptance Criteria:

- [ ] Tracking numbers validate correctly
- [ ] Status updates within 15 minutes
- [ ] Exceptions flagged automatically
- [ ] POD images captured and displayed
- [ ] Carrier API rate limits respected

S4-24: Create Report Builder Interface

Description: Build a flexible report builder for custom reporting needs.

Technical Details:

- Drag-and-drop field selection
- Filter and date range options
- Grouping and aggregation
- Preview before export
- Save as template
- Share with team

Acceptance Criteria:

- [ ] Can select from all available fields
- [ ] Filters apply correctly to data
- [ ] Preview matches final export
- [ ] Templates save and load correctly
- [ ] Export generates valid files

Acceptance Criteria

Sprint Completion Criteria

- [ ] PSP operators can view dashboard with all brands
- [ ] Campaign oversight shows accurate status
- [ ] Shipments can be created and tracked
- [ ] Print queue allows job management
- [ ] Reports can be generated and exported
- [ ] All pages perform well with large datasets

Definition of Done

- [ ] Code reviewed and approved
- [ ] Unit tests with >80% coverage
- [ ] Integration tests for carrier APIs
- [ ] Performance testing with 1000+ records
- [ ] Accessibility audit passed
- [ ] Deployed to staging environment

Dependencies

Incoming Dependencies

From Sprint	Task	Required By	Status
S3	S3-06-10 Campaign Management	S4-06	TBD
S3	S3-15 Store Selection	S4-01	TBD
S3	S3-20-24 Analytics	S4-24	TBD

Outgoing Dependencies

Blocked Task	Blocked Sprint	Impact

S5-02 Campaign Calendar	S5	High - Campaign data
S5-04 External Integrations	S5	High - Shipment data

Risks and Mitigations

Risk	Probability	Impact	Mitigation
Carrier API complexity	High	High	Start integration early
Multi-brand data volume	Medium	High	Pagination, caching
Report builder complexity	Medium	Medium	MVP scope, iterate
Print queue integration	Medium	Medium	Mock initial, real later

---

Team Allocation

Role	Team Member	Capacity	Focus Areas
Tech Lead	TBD	25%	Architecture, reviews
Frontend Dev 1	TBD	100%	Dashboard, campaigns
Frontend Dev 2	TBD	100%	Print queue, reports
Backend Dev 1	TBD	100%	Shipment APIs, carrier integration
Backend Dev 2	TBD	100%	Reporting engine
QA Engineer	TBD	50%	Testing, E2E

Sprint Ceremonies

Ceremony	Date	Time	Duration
Sprint Planning	Mar 3, 2026	10:00 AM	2 hours
Daily Standup	Daily	9:00 AM	15 minutes
Mid-Sprint Review	Mar 7, 2026	2:00 PM	1 hour
Sprint Review	Mar 14, 2026	2:00 PM	1 hour
Sprint Retrospective	Mar 14, 2026	3:30 PM	1 hour

Success Metrics

Metric	Target	Measurement
Dashboard load time	< 3 seconds	Performance monitoring
Campaign list performance	1000+ items smoothly	Manual testing
Shipment tracking accuracy	> 99%	Carrier validation
Report generation time	< 30 seconds	Performance monitoring
Print queue throughput	1000+ items/hour	Load testing

Notes

- Week 9 focus: Dashboard, campaign oversight, multi-brand view
- Week 10 focus: Shipment tracking, print queue, reporting
- Carrier integration requires API keys - obtain early
- Consider WebSocket for real-time shipment updates
- Report builder may need iteration post-MVP

Related Documents

- Sprint Roadmap
- Sprint Calendar
- Dependency Map
- Sprint S3: Brand Portal

Last Updated: 2026-01-01

Sprint S5: Store Portal & Integration

Sprint Overview

Attribute	Value
Sprint Number	S5
Theme	Store Portal & Integration
Start Date	2026-03-17
End Date	2026-03-28
Duration	2 weeks (10 working days)
Sprint Goal	Complete the store-facing experience and integrate all system components

Sprint Goals

1. Build store portal with location-specific content
2. Implement campaign calendar with upcoming installations
3. Create display compliance tracking and scoring
4. Integrate with external systems (ERP, shipping)
5. Build notification center with preferences

Task Breakdown

Task ID	Description	Priority	Estimate	Assignee	Status
S5-01	Create store manager portal dashboard	Critical	8h	TBD	Not Started
S5-02	Build store information header	High	4h	TBD	Not Started
S5-03	Implement location-specific campaign display	High	6h	TBD	Not Started
S5-04	Create store performance summary	High	6h	TBD	Not Started
S5-05	Build team member list and management	Medium	6h	TBD	Not Started

S5-06	Create campaign calendar view	Critical	12h	TBD	Not Started
S5-07	Build calendar event details panel	High	6h	TBD	Not Started
S5-08	Implement calendar sync (iCal export)	Medium	4h	TBD	Not Started
S5-09	Create upcoming tasks list widget	High	6h	TBD	Not Started
S5-10	Build installation checklist preview	High	6h	TBD	Not Started
S5-11	Create compliance dashboard	Critical	8h	TBD	Not Started
S5-12	Build compliance score calculation engine	Critical	12h	TBD	Not Started
S5-13	Implement photo verification gallery	High	8h	TBD	Not Started
S5-14	Create compliance history timeline	Medium	6h	TBD	Not Started
S5-15	Build compliance trend charts	Medium	6h	TBD	Not Started
S5-16	Implement ERP integration framework	High	12h	TBD	Not Started
S5-17	Create shipping provider integration	High	10h	TBD	Not Started
S5-18	Build inventory sync module	Medium	8h	TBD	Not Started
S5-19	Implement webhook management UI	Medium	6h	TBD	Not Started
S5-20	Create integration health monitoring	Medium	6h	TBD	Not Started
S5-21	Build notification center UI	Critical	8h	TBD	Not Started
S5-22	Create notification list with filtering	High	6h	TBD	Not Started
S5-23	Implement notification preferences	High	6h	TBD	Not Started
S5-24	Build email notification templates	High	8h	TBD	Not Started

S5-25	Create in-app notification badges	Medium	4h	TBD	Not Started
S5-26	Implement notification digest scheduling	Medium	6h	TBD	Not Started
S5-27	Build store contact information page	Medium	4h	TBD	Not Started
S5-28	Create help and support section	Medium	4h	TBD	Not Started
S5-29	Implement feedback submission form	Low	4h	TBD	Not Started
S5-30	Build FAQ and knowledge base	Low	6h	TBD	Not Started

Task Details

S5-01: Create Store Manager Portal Dashboard

Description: Build the main dashboard for store managers to oversee their location.

Technical Details:

- Store-specific data scope
- Current campaign status cards
- Upcoming deadlines widget
- Team activity summary
- Performance metrics vs. targets
- Quick action buttons

Acceptance Criteria:

- [ ] Dashboard shows only relevant store data
- [ ] Campaign cards link to detail pages
- [ ] Deadlines sorted by urgency
- [ ] Performance compared to targets
- [ ] Page loads within 3 seconds

S5-06: Create Campaign Calendar View

Description: Build a calendar interface showing campaign timelines and installations.

Technical Details:

- Month, week, day view options

- Campaign events with color coding by status
- Drag-and-drop rescheduling (if permitted)
- Click for event details
- Today indicator
- Navigation with keyboard support

Acceptance Criteria:

- [ ] Calendar displays all assigned campaigns
- [ ] Views switch smoothly
- [ ] Events show correct dates
- [ ] Detail panel shows campaign info
- [ ] iCal export generates valid file

S5-12: Build Compliance Score Calculation Engine

Description: Implement the algorithm that calculates store compliance scores.

Technical Details:

- Score components: photo verification, checklist completion, timeliness
- Weighted scoring formula
- Historical trend calculation
- Benchmark comparisons
- Score caching for performance
- Recalculation triggers

Acceptance Criteria:

- [ ] Score formula documented and approved
- [ ] Calculation completes in < 1 second
- [ ] Scores update when new data arrives
- [ ] Historical data preserved
- [ ] Benchmarks calculated correctly

S5-16: Implement ERP Integration Framework

Description: Create the framework for integrating with external ERP systems.

Technical Details:

- Plugin architecture for different ERPs
- OAuth 2.0 authentication flow
- Data mapping configuration
- Sync scheduling
- Error handling and retry logic
- Audit logging

Acceptance Criteria:

- [ ] Framework supports multiple ERP types
- [ ] Authentication works with test ERP
- [ ] Data syncs correctly both directions
- [ ] Errors logged with context
- [ ] Retry logic handles transient failures

Acceptance Criteria

Sprint Completion Criteria

- [ ] Store managers can access portal within 3 seconds
- [ ] Calendar syncs with campaign deadlines
- [ ] Compliance scores calculate accurately
- [ ] At least one external integration working
- [ ] Notification center fully functional
- [ ] All integrations pass validation tests

Definition of Done

- [ ] Code reviewed and approved
- [ ] Unit tests with >80% coverage
- [ ] Integration tests for external systems
- [ ] E2E tests for critical user flows
- [ ] Accessibility audit passed
- [ ] Deployed to staging environment

Dependencies

Incoming Dependencies

From Sprint	Task	Required By	Status
S2	S2-01 Store Dashboard	S5-01	TBD
S2	S2-17 Checklist Execution	S5-11	TBD
S4	S4-06 Campaign Oversight	S5-06	TBD
S4	S4-14 Shipment Tracking	S5-17	TBD

Outgoing Dependencies

Blocked Task	Blocked Sprint	Impact
S6-01 E2E Testing	S6	Critical - All features needed
S6-03 Security Audit	S6	High - Integrations needed

Risks and Mitigations

Risk	Probability	Impact	Mitigation
ERP integration complexity	High	High	Start with one ERP type
Compliance calculation accuracy	Medium	High	Thorough testing, UAT
Calendar performance	Medium	Medium	Lazy loading, virtualization
Notification delivery reliability	Medium	Medium	Retry logic, fallbacks

---

Team Allocation

Role	Team Member	Capacity	Focus Areas
Tech Lead	TBD	25%	Architecture, integrations
Frontend Dev 1	TBD	100%	Portal, calendar
Frontend Dev 2	TBD	100%	Compliance, notifications
Backend Dev 1	TBD	100%	ERP integration
Backend Dev 2	TBD	100%	Compliance engine, notifications
Mobile Dev	TBD	50%	Mobile notification sync
QA Engineer	TBD	50%	Testing, E2E

Sprint Ceremonies

Ceremony	Date	Time	Duration
Sprint Planning	Mar 17, 2026	10:00 AM	2 hours
Daily Standup	Daily	9:00 AM	15 minutes
Mid-Sprint Review	Mar 21, 2026	2:00 PM	1 hour
Sprint Review	Mar 28, 2026	2:00 PM	1 hour

Sprint Retrospective	Mar 28, 2026	3:30 PM	1 hour

Success Metrics

Metric	Target	Measurement
Portal load time	< 3 seconds	Performance monitoring
Calendar navigation	< 500ms	Performance monitoring
Compliance score accuracy	> 99%	Manual validation
Integration sync success	> 99%	Error rate monitoring
Notification delivery	> 99%	Delivery tracking

Notes

- Week 11 focus: Portal, calendar, notifications
- Week 12 focus: Compliance tracking, external integrations
- ERP integration may need vendor coordination
- Compliance formula needs business sign-off
- Plan integration testing environment setup

Related Documents

- Sprint Roadmap
- Sprint Calendar
- Dependency Map
- Sprint S4: PSP Portal

Last Updated: 2026-01-01

Sprint S6: Polish, Testing & Beta Launch

Sprint Overview

Attribute	Value
Sprint Number	S6
Theme	Polish, Testing & Beta Launch
Start Date	2026-03-31
End Date	2026-04-04
Duration	1 week (5 working days)
Sprint Goal	Ensure production readiness and launch beta program

Sprint Goals

1. Execute comprehensive end-to-end testing across all portals
2. Optimize performance and implement caching strategies
3. Complete security audit and penetration testing
4. Finalize documentation and training materials
5. Launch beta program with select partners

Task Breakdown

Task ID	Description	Priority	Estimate	Assignee	Status
S6-01	Execute E2E tests - Brand Portal flows	Critical	8h	TBD	Not Started
S6-02	Execute E2E tests - PSP Portal flows	Critical	8h	TBD	Not Started
S6-03	Execute E2E tests - Store Portal flows	Critical	6h	TBD	Not Started
S6-04	Execute E2E tests - Mobile App flows	Critical	8h	TBD	Not Started
S6-05	Execute cross-portal integration tests	Critical	6h	TBD	Not Started

S6-06	Bug triage and critical fix sprint	Critical	12h	TBD	Not Started
S6-07	Implement API response caching	High	6h	TBD	Not Started
S6-08	Optimize database query performance	High	8h	TBD	Not Started
S6-09	Implement CDN for static assets	High	4h	TBD	Not Started
S6-10	Add lazy loading for large components	Medium	4h	TBD	Not Started
S6-11	Optimize image loading and compression	High	4h	TBD	Not Started
S6-12	Run performance benchmark tests	High	4h	TBD	Not Started
S6-13	Execute automated security scan	Critical	4h	TBD	Not Started
S6-14	Perform manual penetration testing	Critical	8h	TBD	Not Started
S6-15	Review and fix security findings	Critical	8h	TBD	Not Started
S6-16	Verify data encryption implementation	High	2h	TBD	Not Started
S6-17	Audit access control configurations	High	4h	TBD	Not Started
S6-18	Complete API documentation	High	6h	TBD	Not Started
S6-19	Finalize user guides per portal	High	8h	TBD	Not Started
S6-20	Create admin operations runbook	High	4h	TBD	Not Started
S6-21	Record video tutorials	Medium	6h	TBD	Not Started
S6-22	Prepare release notes	High	2h	TBD	Not Started
S6-23	Configure production environment	Critical	4h	TBD	Not Started
S6-24	Deploy to production infrastructure	Critical	4h	TBD	Not Started

S6-25	Set up monitoring and alerting	Critical	4h	TBD	Not Started
S6-26	Configure backup and recovery	High	2h	TBD	Not Started
S6-27	Onboard beta partner accounts	High	4h	TBD	Not Started
S6-28	Conduct beta partner training	High	4h	TBD	Not Started
S6-29	Establish support escalation process	High	2h	TBD	Not Started
S6-30	Go-live checklist verification	Critical	2h	TBD	Not Started

---

Task Details

S6-01: Execute E2E Tests - Brand Portal Flows

Description: Run comprehensive end-to-end tests covering all Brand Portal functionality.

Test Scenarios:

1. Brand admin login and dashboard access
2. Complete campaign creation wizard
3. Store selection and filtering
4. Analytics dashboard data accuracy
5. Asset library upload and management
6. Team member invitation flow
7. Campaign duplication and archiving
8. Report generation and export

Acceptance Criteria:

- [ ] All test scenarios pass
- [ ] No critical or high severity bugs
- [ ] Test coverage report generated
- [ ] Performance within acceptable limits
- [ ] Accessibility compliance verified

S6-08: Optimize Database Query Performance

Description: Review and optimize database queries for performance.

Technical Details:

- Analyze slow query logs
- Add missing indexes
- Optimize N+1 query patterns
- Implement query result caching
- Review connection pooling
- Test with production-scale data

Acceptance Criteria:

- [ ] No queries > 500ms in typical usage
- [ ] Dashboard queries < 200ms
- [ ] List queries with pagination optimal
- [ ] Connection pool sized correctly
- [ ] Load test passes with 100 concurrent users

S6-14: Perform Manual Penetration Testing

Description: Conduct manual security testing to identify vulnerabilities.

Testing Areas:

1. Authentication bypass attempts
2. Authorization testing (role escalation)
3. SQL injection testing
4. XSS vulnerability testing
5. CSRF protection verification
6. API security testing
7. File upload security
8. Session management

Acceptance Criteria:

- [ ] No critical vulnerabilities
- [ ] No high-severity vulnerabilities
- [ ] All medium findings documented with remediation plan
- [ ] Penetration test report delivered
- [ ] Remediation verified

S6-24: Deploy to Production Infrastructure

Description: Deploy the complete application to production environment.

Deployment Steps:

1. Database migration execution

2. Backend service deployment
3. Frontend application deployment
4. Mobile app submission to stores
5. CDN cache warming
6. DNS configuration
7. SSL certificate verification
8. Health check confirmation

Acceptance Criteria:

- [ ] All services deployed successfully
- [ ] Health checks passing
- [ ] No 5xx errors in first hour
- [ ] Monitoring dashboards populated
- [ ] Rollback procedure tested

Acceptance Criteria

Sprint Completion Criteria

- [ ] All E2E tests pass for all portals
- [ ] No critical or high-severity bugs open
- [ ] Page load times under 2 seconds
- [ ] Zero critical security vulnerabilities
- [ ] All documentation complete
- [ ] Production environment deployed
- [ ] Beta partners onboarded
- [ ] 95% uptime achieved during beta

Definition of Done

- [ ] All test cases executed and passed
- [ ] Performance benchmarks met
- [ ] Security audit signed off
- [ ] Documentation reviewed and approved
- [ ] Beta partner training complete
- [ ] Production monitoring active

Dependencies

Incoming Dependencies

From Sprint	Task	Required By	Status

S5	All S5 tasks	S6-01 - S6-05	TBD
S2	S2-05 Push Notifications	S6-01	TBD
S3	S3-05 Asset Library	S6-01	TBD
S4	S4-04 Print Queue	S6-02	TBD
S5	S5-04 External Integrations	S6-13-17	TBD

Outgoing Dependencies

Milestone	Date	Dependency
Beta Launch	Apr 6, 2026	All S6 tasks complete

Daily Schedule

Day 1 - Monday, March 31

Time	Activity	Owner
9:00 AM	Sprint kickoff	Team
10:00 AM	E2E test execution begins	QA
2:00 PM	Performance optimization	Dev
4:00 PM	Security scan initiated	DevOps

Day 2 - Tuesday, April 1

Time	Activity	Owner
9:00 AM	Bug triage	Team
10:00 AM	Critical bug fixes	Dev
2:00 PM	Performance benchmark	DevOps
4:00 PM	E2E test completion	QA

Day 3 - Wednesday, April 2

Time	Activity	Owner
9:00 AM	Penetration testing	Security
10:00 AM	Security fix sprint	Dev

2:00 PM	Documentation review	Tech Lead
4:00 PM	Security sign-off	Security

Day 4 - Thursday, April 3

Time	Activity	Owner
9:00 AM	Production config	DevOps
10:00 AM	Final bug fixes	Dev
2:00 PM	User guide finalization	PM
4:00 PM	Deployment rehearsal	DevOps

Day 5 - Friday, April 4

Time	Activity	Owner
9:00 AM	Production deployment	DevOps
11:00 AM	Smoke testing	QA
1:00 PM	Beta partner onboarding	PM
3:00 PM	Beta partner training	Team
5:00 PM	Go-live checklist	Team

Risks and Mitigations

Risk	Probability	Impact	Mitigation
Critical bugs discovered	High	Critical	Buffer time for fixes
Performance issues	Medium	High	Pre-optimization in prior sprints
Security findings	Medium	Critical	Early security testing
Deployment failures	Low	Critical	Rollback procedure ready
Beta partner availability	Low	Medium	Early scheduling

Team Allocation

Role	Team Member	Focus Areas
Tech Lead	TBD	Architecture, reviews, sign-offs

Frontend Dev	TBD	Bug fixes, optimization
Backend Dev	TBD	Bug fixes, performance
DevOps	TBD	Deployment, infrastructure
QA Lead	TBD	E2E testing, regression
Security Engineer	TBD	Penetration testing
PM	TBD	Documentation, beta coordination

Go-Live Checklist

Pre-Deployment

- [ ] All E2E tests passing
- [ ] Performance benchmarks met
- [ ] Security audit complete
- [ ] Documentation finalized
- [ ] Release notes prepared
- [ ] Stakeholder sign-off obtained

Deployment

- [ ] Database backups verified
- [ ] Migration scripts tested
- [ ] Deployment runbook followed
- [ ] Health checks passing
- [ ] Monitoring active

Post-Deployment

- [ ] Smoke tests executed
- [ ] Beta accounts activated
- [ ] Support team briefed
- [ ] Feedback channels open
- [ ] Rollback tested

Success Metrics

Metric	Target	Measurement
E2E test pass rate	100%	Test reports

Page load time	< 2 seconds	Lighthouse
Security vulnerabilities	0 critical	Security report
Documentation coverage	100%	Manual review
Beta uptime	> 95%	Monitoring

Notes

- Compressed 1-week sprint - all hands on deck
- Daily check-ins at 9 AM and 4 PM
- Escalation path to leadership for blockers
- Weekend availability for critical issues
- Beta launch Monday April 6, 2026

Related Documents

- Sprint Roadmap
- Sprint Calendar
- Dependency Map
- Sprint S5: Store Integration

Last Updated: 2026-01-01

AI Development Harness Strategy

> Defines the "Agentic" methodology for building NewPOPSys v1, leveraging the `SOW` architecture as the primary context.

---

1. The Orchestrator Model

We use a **Dispatcher/Worker** model to manage the complexity of the full stack build.

1.1 The Dispatcher (Plan & Verify)

The User acts as the Architect, but the **Orchestrator Agent** (You) manages the Sprint Board.

- **Rule**: Never edit code directly in Orchestrator Mode.
- **Responsibility**: Read `CURRENT_SPRINT.md`, identify the next task, and Dispatch a Worker.
- **Context**: Loads `MASTER_DEVELOPMENT_PLAN.md` and `TECH_STACK_DECISIONS.md`.

1.2 The Worker (Execute)

A "Worker" is a targeted agent session focused on a **Single Task**.

- **Rule**: Only load the files necessary for the specific task.
- **Responsibility**: Implement the feature, run tests, and report back.
- **Context**: Loads the specific `SUPP-0xx` relevant to the feature.

---

2. Context Loading Strategy

To avoid context window overflow, we strictly map Modules to SUPP files.

Module	Primary Context	Supplemental Context
Core/Auth	`SUPP-003_RBAC`	`TECH_STACK_DECISIONS.md`
Brand Admin	`SUPP-015_Campaigns`	`SUPP-013_Stores`
PSP Ops	`SUPP-016_Orders`	`SUPP-006_Webhooks`
Mobile App	`SUPP-017_Execution`	`SUPP-011_Offline`
API/Backend	`SUPP-006_Inbound`	`SUPP-029_Observability`

2.1 The "Context Header"

Every Worker session should start by reading the **Authoritative Spec**:

> "I am working on Task [ID]. Please read `SOW/02_SUPPs/.../SUPP-XXX.md` to understand the requirements."

---

3. Development Workflows

3.1 Feature Implementation Loop

1. **Checkout**: `git checkout -b feat/POP-XXX-description`
2. **Spec Check**: Read the relevant `SUPP` file.
3. **Scaffold**: Create file structures (e.g., `apps/brand-portal/app/campaigns/page.tsx`).
4. **Implement**: Write code using **Next.js** / **Fastify** patterns.
5. **Test**: Write Unit Tests (`vitest`).
6. **Verify**: Run `npm run test` and `npm run lint`.
7. **Commit**: `git commit -m "feat(module): description (POP-XXX)"`

3.2 The "Audit" Gate

Before marking a task as Done, the Orchestrator must run an **Audit**:

1. **Check Requirements**: Does the code match `SUPP` Acceptance Criteria?
2. **Check Stack**: Did we use Next.js/Fastify, or accidentally slip into Express/Vite?
3. **Check Lint**: Are there `eslint` errors?

---

4. Automation Scripts (The Harness)

We will build a local CLI `scripts/dev-harness.ts` to assist the AI.

4.1 `npm run ai:context`

Generates a "Context Pack" for the current task.

- Reads `CURRENT_SPRINT.md`.
- identifying active task.
- Concatenates relevant `SUPP` files into a single buffer for the AI to read.

4.2 `npm run ai:verify`

Runs a strict suite of checks:

- TypeCheck (TSC)
- Lint (ESLint)

- Test (Vitest)
- Boundary Check (Ensures no illegal imports between Monorepo packages).

---

5. Agent Instructions (System Prompt Injection)

When initializing a Worker, include this specific instruction:

```
You are a Senior Full Stack Engineer building NewPOPSys v1.
STACK: Turborepo, Next.js (Web), Fastify (API), Postgres (Drizzle).
SOURCE OF TRUTH: SOW/02_SUPPs/*.md.
RULE: Do not invent features. If it's not in the SUPP, ask the Orchestrator.
```

Last Updated: 2026-01-02

Technology Stack Decisions

This document records architectural decisions using the ADR (Architecture Decision Record) format.

ADR-001: React 18 with TypeScript

Status

Accepted

Context

We need to select a frontend framework for building four distinct applications:

- Mobile PWA (field reps)
- Brand Admin Dashboard
- PSP Operations Center
- Store Portal

Key requirements:

- Cross-platform mobile support via PWA
- Complex state management
- Real-time updates capability
- Strong typing for large codebase
- Developer ecosystem and hiring pool

Decision

We will use **Next.js 14+ (App Router)** with **TypeScript 5.3+** as our frontend framework.

Alternatives Considered

Framework	Pros	Cons
Vite + React	Lightweight, purely client-side	Manual routing/SSR setup needed
Next.js	Full framework, Server Components, SSR, API routes	Heavier, opinionated router
Remix	Great data loading patterns	Smaller ecosystem than Next.js

Consequences

Positive:

- Unified framework for UI and API (if needed)
- Server Components allow smaller client bundles
- Excellent TypeScript integration
- Strong PWA support with next-pwa
- Large talent pool for hiring
- Vercel backing ensures stability

Negative:

- Higher complexity than plain React
- "Use Client" directives learning curve

ADR-002: Fastify over Express

Status

Accepted

Context

We need a Node.js web framework for our API server that handles:

- High request throughput (thousands of field reps)
- Photo upload processing
- Real-time task updates
- Complex validation

Decision

We will use **Fastify 4.x** as our backend web framework.

Alternatives Considered

Framework	Throughput (req/s)	Pros	Cons
Express	~15,000	Mature, widespread	Slower, callback-based design
Fastify	~75,000	5x faster, schema-first, plugins	Smaller community
Koa	~20,000	Lightweight, modern	Less ecosystem support

Hono	~100,000+	Extremely fast, edge-ready	Very new, less mature

Consequences

Positive:

- 5x performance improvement over Express
- Built-in schema validation with JSON Schema
- Plugin architecture aligns with our modular design
- Excellent TypeScript support
- Request/response serialization

Negative:

- Smaller middleware ecosystem than Express
- Team needs to learn Fastify patterns
- Some Express middleware needs adaptation

Benchmarks Reference

```
Framework      Requests/sec
-----------    ------------
Fastify        76,835
Koa            22,497
Express        15,123
```

ADR-003: Drizzle ORM over Prisma

Status

Accepted

Context

We need an ORM/query builder for PostgreSQL that provides:

- Type-safe database queries
- Migration management
- Good performance for complex queries
- JSON/JSONB support for flexible data

Decision

We will use **Drizzle ORM 0.29+** as our database toolkit.

Alternatives Considered

ORM	Type Safety	Performance	Flexibility
Prisma	Excellent	Good (query engine overhead)	Limited (abstraction-heavy)
Drizzle	Excellent	Excellent (SQL-like)	High (thin abstraction)
TypeORM	Good	Moderate	Moderate
Knex	Manual	Excellent	High

Consequences

Positive:

- SQL-like syntax familiar to database developers
- No query engine binary (smaller deployments)
- Type-safe joins and relations
- Excellent PostgreSQL JSONB support
- Relational queries without N+1 issues
- Lightweight bundle size

Negative:

- Less mature than Prisma
- Smaller community and fewer tutorials
- Manual migration writing (vs Prisma's auto-generate)

Code Comparison

```
// Drizzle - SQL-like, transparent
const tasks = await db
  .select()
  .from(tasksTable)
  .leftJoin(storesTable, eq(tasksTable.storeId, storesTable.id))
  .where(eq(tasksTable.status, 'pending'))
  .limit(10);

// Prisma - More abstracted
const tasks = await prisma.task.findMany({
  where: { status: 'pending' },
  include: { store: true },
  take: 10,
});
```

ADR-004: PostgreSQL 16

Status

Accepted

Context

We need a primary database that supports:

- Complex relational queries (campaigns, stores, tasks)
- JSON/JSONB for flexible configurations
- Geospatial queries (store locations)
- High reliability and ACID compliance
- Horizontal read scaling

Decision

We will use **PostgreSQL 16** as our primary database.

Alternatives Considered

Database	JSON Support	Geo Support	Scalability	Cost
PostgreSQL	Excellent (JSONB)	PostGIS	Read replicas	Free
MySQL 8	Good (JSON)	Basic	Read replicas	Free
MongoDB	Native	Native	Horizontal	Free/Paid
CockroachDB	Good	Limited	Distributed	Paid

Consequences

Positive:

- PostgreSQL 16's improved JSON performance
- JSONB for flexible POP kit configurations
- PostGIS extension for store proximity queries
- Mature, battle-tested reliability
- Excellent Drizzle ORM integration
- Free and open source

Negative:

- Vertical scaling limitations
- Need to manage read replicas manually
- More complex setup than managed NoSQL

PostgreSQL 16 Features We'll Use

- **JSONB path queries** - For querying nested POP kit configs
- **Parallel query execution** - For analytics dashboards

- **Improved vacuum** - Better performance under load
- **pg_stat_io** - Better monitoring capabilities

ADR-005: Turborepo for Monorepo

Status

Accepted

Context

We have multiple applications (4 frontend apps, 1 API) and shared packages that need:

- Unified dependency management
- Shared code between apps
- Consistent build and test pipelines
- Incremental builds for CI performance

Decision

We will use **Turborepo** as our monorepo build system with **pnpm** workspaces.

Alternatives Considered

Tool	Speed	Learning Curve	Features
Turborepo	Excellent	Low	Caching, pipelines
Nx	Excellent	High	Full-featured, complex
Lerna	Moderate	Low	Publishing focus
Rush	Good	High	Enterprise focus

Consequences

Positive:

- Zero-config setup for most cases
- Excellent build caching (local + remote)
- Simple pipeline configuration
- Works seamlessly with pnpm
- Lower learning curve than Nx
- Vercel backing ensures maintenance

Negative:

- Fewer features than Nx
- No built-in generators/schematics
- Less granular task dependencies

Configuration

```
// turbo.json
{
  "$schema": "https://turbo.build/schema.json",
  "pipeline": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**", ".next/**"]
    },
    "dev": {
      "cache": false,
      "persistent": true
    },
    "lint": {},
    "test": {
      "dependsOn": ["build"]
    }
  }
}
```

ADR-006: TanStack Query for Data Fetching

Status

Accepted

Context

We need a data fetching solution that handles:

- Server state caching
- Background refetching
- Optimistic updates for offline-first mobile
- Request deduplication
- Pagination and infinite loading

Decision

We will use **TanStack Query (React Query) 5.x** for server state management.

Alternatives Considered

Library	Caching	Offline	DX	Bundle Size
TanStack Query	Excellent	Good	Excellent	12kb
SWR	Good	Limited	Good	4kb
RTK Query	Good	Limited	Moderate	Redux dependency
Apollo Client	Excellent	Good	Good	35kb+

Consequences

Positive:

- Automatic caching and invalidation
- Built-in loading/error states
- Optimistic updates for responsive UI
- Query invalidation by key patterns
- Excellent DevTools
- Mutation hooks for write operations

Negative:

- Additional learning curve
- Need to design cache key strategy
- Potential for stale data if not configured properly

Usage Pattern

```
// hooks/useTasks.ts
export function useTasks(campaignId: string) {
  return useQuery({
    queryKey: ['tasks', { campaignId }],
    queryFn: () => api.tasks.list({ campaignId }),
    staleTime: 5 * 60 * 1000, // 5 minutes
  });
}

export function useCompleteTask() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: api.tasks.complete,
    onSuccess: (_, variables) => {
      queryClient.invalidateQueries({
        queryKey: ['tasks', { taskId: variables.taskId }]
      });
    },
  });
}
```

Summary Matrix

Decision	Choice	Primary Reason
Frontend Framework	React 18 + TypeScript	Ecosystem & PWA support
Backend Framework	Fastify 4	Performance (5x Express)
ORM	Drizzle	Type-safety + SQL control
Database	PostgreSQL 16	JSONB + reliability
Monorepo	Turborepo	Simplicity + caching
Data Fetching	TanStack Query	Caching + offline support

Last Updated: 2026-01-01

AI Development Specifications

Target Audience

This document is designed for **AI coding assistants** (Claude, Cursor, GitHub Copilot, and similar tools) to provide context for code generation, review, and assistance within the PopSystem project.

Technology Stack

Frontend

Technology	Version	Purpose
Next.js	14+	App Router & React Framework
TypeScript	5.3+	Type-safe JavaScript
Turbo	Latest	Build tool and dev server
TanStack Query	5.x	Server state management
React Hook Form	7.x	Form handling
Zod	3.x	Schema validation
TailwindCSS	3.4+	Utility-first CSS
Radix UI	Latest	Accessible component primitives

Backend

Technology	Version	Purpose
Node.js	20 LTS	Runtime environment
Fastify	4.x	High-performance web framework
TypeScript	5.3+	Type-safe JavaScript
Drizzle ORM	0.29+	Type-safe SQL ORM
PostgreSQL	16	Primary database
Redis	7.x	Caching and sessions

Infrastructure

Technology	Purpose
Turborepo	Monorepo build system
Docker	Containerization
pnpm	Package manager
Vitest	Unit testing
Playwright	E2E testing

Turborepo Structure

```
popsystem/
├── apps/
│   ├── mobile-pwa/          # Field rep mobile application
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── pages/
│   │   │   ├── hooks/
│   │   │   ├── stores/
│   │   │   └── utils/
│   │   ├── public/
│   │   └── package.json
│   │
│   ├── brand-admin/         # Brand manager dashboard
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── pages/
│   │   │   ├── hooks/
│   │   │   └── stores/
│   │   └── package.json
│   │
│   ├── psp-ops/             # PSP operations center
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── pages/
│   │   │   ├── hooks/
│   │   │   └── stores/
│   │   └── package.json
│   │
│   ├── store-portal/        # Store manager interface
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── pages/
│   │   │   ├── hooks/
│   │   │   └── stores/
│   │   └── package.json
│   │
│   └── api/                 # Backend API server
│       ├── src/
│       │   ├── routes/
│       │   ├── services/
│       │   ├── middleware/
│       │   └── plugins/
│       └── package.json
```

```
├── packages/
│   ├── ui/                    # Shared UI components
│   ├── api-client/            # Generated API client
│   ├── shared-types/          # Shared TypeScript types
│   ├── database/              # Drizzle schema and migrations
│   ├── config-eslint/         # Shared ESLint config
│   └── config-typescript/     # Shared TS config
│
├── turbo.json
├── pnpm-workspace.yaml
└── package.json
```

Shared UI Components Strategy

Component Library Location

All shared components live in `packages/ui/` following atomic design principles:

```
packages/ui/
├── src/
│   ├── atoms/             # Basic building blocks
│   │   ├── Button/
│   │   ├── Input/
│   │   ├── Badge/
│   │   └── Icon/
│   │
│   ├── molecules/         # Combinations of atoms
│   │   ├── FormField/
│   │   ├── Card/
│   │   ├── SearchInput/
│   │   └── StatusBadge/
│   │
│   ├── organisms/         # Complex components
│   │   ├── DataTable/
│   │   ├── FormBuilder/
│   │   ├── PhotoCapture/
│   │   └── TaskList/
│   │
│   └── templates/         # Page layouts
│       ├── DashboardLayout/
│       ├── FormLayout/
│       └── MobileLayout/
│
├── styles/
│   └── globals.css
└── index.ts
```

Component Export Pattern

```typescript
// packages/ui/src/atoms/Button/index.ts
export { Button } from './Button';
export type { ButtonProps } from './Button.types';
```

```
// packages/ui/index.ts
export * from './src/atoms/Button';
export * from './src/molecules/FormField';
export * from './src/organisms/DataTable';
```

Usage in Apps

```
// apps/mobile-pwa/src/components/TaskCard.tsx
import { Button, Card, StatusBadge } from '@popsystem/ui';
```

API Patterns

RESTful Conventions

Method	Endpoint Pattern	Purpose
GET	`/api/v1/{resource}`	List resources
GET	`/api/v1/{resource}/:id`	Get single resource
POST	`/api/v1/{resource}`	Create resource
PATCH	`/api/v1/{resource}/:id`	Partial update
PUT	`/api/v1/{resource}/:id`	Full replacement
DELETE	`/api/v1/{resource}/:id`	Delete resource

Request Schema Pattern

```
// Zod schema for request validation
const createTaskSchema = z.object({
  body: z.object({
    campaignId: z.string().uuid(),
    storeId: z.string().uuid(),
    type: z.enum(['installation', 'verification', 'removal']),
    scheduledDate: z.string().datetime(),
    instructions: z.string().optional(),
  }),
  params: z.object({}),
  query: z.object({}),
});

type CreateTaskRequest = z.infer<typeof createTaskSchema>;
```

Response Schema Pattern

```ts
// Standard success response
interface ApiResponse<T> {
  success: true;
  data: T;
  meta?: {
    page?: number;
    pageSize?: number;
    total?: number;
  };
}

// Standard error response
interface ApiError {
  success: false;
  error: {
    code: string;
    message: string;
    details?: Record<string, string[]>;
  };
}

// Paginated response
interface PaginatedResponse<T> extends ApiResponse<T[]> {
  meta: {
    page: number;
    pageSize: number;
    total: number;
    totalPages: number;
  };
}
```

Route Handler Pattern

```ts
// apps/api/src/routes/tasks/create.ts
import { FastifyInstance } from 'fastify';
import { createTaskSchema } from './schemas';

export async function createTaskRoute(fastify: FastifyInstance) {
  fastify.post(
    '/tasks',
    {
      schema: {
        body: createTaskSchema.shape.body,
        response: {
          201: taskResponseSchema,
          400: errorResponseSchema,
        },
      },
      preHandler: [fastify.authenticate],
    },
    async (request, reply) => {
      const task = await fastify.taskService.create(request.body);
      return reply.status(201).send({ success: true, data: task });
    }
```

```
  );
}
```

Database Schema Overview

Core Tables

Users

```
// packages/database/src/schema/users.ts
export const users = pgTable('users', {
  id: uuid('id').defaultRandom().primaryKey(),
  email: varchar('email', { length: 255 }).notNull().unique(),
  passwordHash: varchar('password_hash', { length: 255 }),
  role: userRoleEnum('role').notNull(),
  firstName: varchar('first_name', { length: 100 }),
  lastName: varchar('last_name', { length: 100 }),
  phone: varchar('phone', { length: 20 }),
  organizationId: uuid('organization_id').references(() => organizations.id)
  isActive: boolean('is_active').default(true),
  lastLoginAt: timestamp('last_login_at'),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});
```

Campaigns

```
export const campaigns = pgTable('campaigns', {
  id: uuid('id').defaultRandom().primaryKey(),
  brandId: uuid('brand_id').references(() => brands.id).notNull(),
  name: varchar('name', { length: 255 }).notNull(),
  description: text('description'),
  status: campaignStatusEnum('status').default('draft'),
  startDate: date('start_date'),
  endDate: date('end_date'),
  budget: decimal('budget', { precision: 10, scale: 2 }),
  targetStoreCount: integer('target_store_count'),
  popKitConfig: jsonb('pop_kit_config'),
  createdBy: uuid('created_by').references(() => users.id),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});
```

Stores

```
export const stores = pgTable('stores', {
  id: uuid('id').defaultRandom().primaryKey(),
  retailerId: uuid('retailer_id').references(() => retailers.id),
```

```
  storeNumber: varchar('store_number', { length: 50 }),
  name: varchar('name', { length: 255 }).notNull(),
  address: varchar('address', { length: 500 }),
  city: varchar('city', { length: 100 }),
  state: varchar('state', { length: 50 }),
  zipCode: varchar('zip_code', { length: 20 }),
  latitude: decimal('latitude', { precision: 10, scale: 7 }),
  longitude: decimal('longitude', { precision: 10, scale: 7 }),
  timezone: varchar('timezone', { length: 50 }),
  storeMetadata: jsonb('store_metadata'),
  isActive: boolean('is_active').default(true),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});
```

Tasks

```
export const tasks = pgTable('tasks', {
  id: uuid('id').defaultRandom().primaryKey(),
  campaignId: uuid('campaign_id').references(() => campaigns.id).notNull(),
  storeId: uuid('store_id').references(() => stores.id).notNull(),
  assignedTo: uuid('assigned_to').references(() => users.id),
  type: taskTypeEnum('type').notNull(),
  status: taskStatusEnum('status').default('pending'),
  priority: integer('priority').default(5),
  scheduledDate: date('scheduled_date'),
  completedAt: timestamp('completed_at'),
  instructions: text('instructions'),
  notes: text('notes'),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});
```

Compliance Records

```
export const complianceRecords = pgTable('compliance_records', {
  id: uuid('id').defaultRandom().primaryKey(),
  taskId: uuid('task_id').references(() => tasks.id).notNull(),
  submittedBy: uuid('submitted_by').references(() => users.id).notNull(),
  status: complianceStatusEnum('status').default('pending_review'),
  submittedAt: timestamp('submitted_at').defaultNow(),
  reviewedBy: uuid('reviewed_by').references(() => users.id),
  reviewedAt: timestamp('reviewed_at'),
  score: integer('score'),
  feedback: text('feedback'),
  checklistData: jsonb('checklist_data'),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});

export const compliancePhotos = pgTable('compliance_photos', {
  id: uuid('id').defaultRandom().primaryKey(),
  complianceRecordId: uuid('compliance_record_id').references(() => complian
  photoUrl: varchar('photo_url', { length: 500 }).notNull(),
```

```
  photoType: photoTypeEnum('photo_type'),
  caption: varchar('caption', { length: 255 }),
  metadata: jsonb('metadata'),
  uploadedAt: timestamp('uploaded_at').defaultNow(),
});
```

Enum Definitions

```
// packages/database/src/schema/enums.ts
export const userRoleEnum = pgEnum('user_role', [
  'super_admin',
  'brand_admin',
  'psp_admin',
  'psp_manager',
  'field_rep',
  'store_manager',
]);

export const campaignStatusEnum = pgEnum('campaign_status', [
  'draft',
  'pending_approval',
  'approved',
  'active',
  'paused',
  'completed',
  'cancelled',
]);

export const taskStatusEnum = pgEnum('task_status', [
  'pending',
  'assigned',
  'in_progress',
  'completed',
  'verified',
  'failed',
  'cancelled',
]);

export const taskTypeEnum = pgEnum('task_type', [
  'installation',
  'verification',
  'maintenance',
  'removal',
  'survey',
]);

export const complianceStatusEnum = pgEnum('compliance_status', [
  'pending_review',
  'approved',
  'rejected',
  'needs_revision',
]);
```

Code Generation Guidelines for AI Assistants

When generating React components:

1. Use functional components with TypeScript
2. Define props interface separately
3. Use Tailwind CSS for styling
4. Export from index.ts file
5. Include proper accessibility attributes

When generating API routes:

1. Use Zod for request/response validation
2. Follow RESTful conventions
3. Include proper error handling
4. Use dependency injection for services
5. Include authentication middleware

When generating database code:

1. Use Drizzle ORM syntax
2. Include proper indexes
3. Define relations explicitly
4. Use appropriate column types
5. Include timestamps on all tables

Last Updated: 2026-01-01

Architecture Overview

This document provides a high-level view of the PopSystem architecture with visual diagrams.

System Context

```
C4Context
    title System Context Diagram - PopSystem

    Person(fieldRep, "Field Rep", "Installs and verifies POP displays")
    Person(brandAdmin, "Brand Admin", "Manages campaigns and reviews complia
    Person(pspOps, "PSP Operations", "Coordinates logistics and assignments"
    Person(storeManager, "Store Manager", "Approves installations, reports i

    System(popSystem, "PopSystem", "POP Display Management Platform")

    System_Ext(imageStorage, "Cloud Storage", "Photo storage (S3/GCS)")
    System_Ext(notifications, "Notification Service", "Push/SMS/Email")
    System_Ext(maps, "Maps API", "Geocoding and routing")

    Rel(fieldRep, popSystem, "Uses", "Mobile PWA")
    Rel(brandAdmin, popSystem, "Manages", "Web Dashboard")
    Rel(pspOps, popSystem, "Coordinates", "Web Dashboard")
    Rel(storeManager, popSystem, "Reviews", "Web Portal")

    Rel(popSystem, imageStorage, "Stores photos")
    Rel(popSystem, notifications, "Sends alerts")
    Rel(popSystem, maps, "Geocodes stores")
```

High-Level Architecture

```
graph TB
    subgraph "Client Applications"
        Mobile["Mobile PWA<br/>(Field Reps)"]
        Brand["Brand Admin<br/>(Dashboard)"]
        PSP["PSP Ops<br/>(Operations)"]
        Store["Store Portal<br/>(Managers)"]
    end

    subgraph "Edge/CDN"
        CDN["Cloudflare/Vercel Edge"]
    end

    subgraph "API Layer"
        Gateway["API Gateway<br/>(Rate Limiting, Auth)"]
        API["Fastify API Server"]
    end
```

```
subgraph "Services"
    Auth["Auth Service"]
    Tasks["Task Service"]
    Campaigns["Campaign Service"]
    Compliance["Compliance Service"]
    Analytics["Analytics Service"]
end

subgraph "Data Layer"
    PG[(PostgreSQL 16)]
    Redis[(Redis Cache)]
    S3["S3/GCS<br/>(Photos)"]
end

subgraph "External Services"
    Push["Push Notifications"]
    SMS["SMS Gateway"]
    Email["Email Service"]
    Maps["Maps API"]
end

Mobile --> CDN
Brand --> CDN
PSP --> CDN
Store --> CDN

CDN --> Gateway
Gateway --> API

API --> Auth
API --> Tasks
API --> Campaigns
API --> Compliance
API --> Analytics

Auth --> PG
Auth --> Redis
Tasks --> PG
Campaigns --> PG
Compliance --> PG
Compliance --> S3
Analytics --> PG

API --> Push
API --> SMS
API --> Email
Tasks --> Maps
```

Application Architecture

Frontend Application Structure

```
graph TB
    subgraph "React Application"
        Pages["Pages/Routes"]
        Features["Feature Components"]
        UI["UI Components<br/>(packages/ui)"]

        subgraph "State Management"
            TQ["TanStack Query<br/>(Server State)"]
            Zustand["Zustand<br/>(Client State)"]
        end

        subgraph "Data Layer"
            APIClient["API Client<br/>(packages/api-client)"]
            Types["Shared Types<br/>(packages/shared-types)"]
        end
    end

    Pages --> Features
    Features --> UI
    Features --> TQ
    Features --> Zustand
    TQ --> APIClient
    APIClient --> Types
```

Backend Service Architecture

```
graph TB
    subgraph "Fastify Application"
        Routes["Route Handlers"]
        Middleware["Middleware Stack"]
        Plugins["Fastify Plugins"]

        subgraph "Business Logic"
            Services["Domain Services"]
            Validators["Zod Validators"]
        end

        subgraph "Data Access"
            Drizzle["Drizzle ORM"]
            Cache["Redis Client"]
            Storage["Storage Client"]
        end
    end

    Routes --> Middleware
    Routes --> Services
    Services --> Validators
    Services --> Drizzle
    Services --> Cache
    Services --> Storage
    Middleware --> Plugins
```

Data Flow Diagrams

Task Completion Flow

```
sequenceDiagram
    participant FR as Field Rep (Mobile)
    participant API as API Server
    participant DB as PostgreSQL
    participant S3 as Photo Storage
    participant WS as WebSocket
    participant Brand as Brand Admin

    FR->>FR: Complete checklist
    FR->>FR: Capture photos

    FR->>API: POST /tasks/:id/complete
    API->>S3: Upload photos
    S3-->>API: Photo URLs

    API->>DB: Create compliance record
    API->>DB: Update task status
    DB-->>API: Confirmation

    API->>WS: Emit task.completed
    API-->>FR: Success response

    WS-->>Brand: Real-time update
    Brand->>Brand: Dashboard updates
```

Campaign Creation Flow

```
sequenceDiagram
    participant BA as Brand Admin
    participant API as API Server
    participant DB as PostgreSQL
    participant PSP as PSP Operations

    BA->>API: POST /campaigns
    API->>DB: Create campaign (draft)
    DB-->>API: Campaign ID
    API-->>BA: Campaign created

    BA->>API: POST /campaigns/:id/submit
    API->>DB: Update status (pending_approval)
    API->>PSP: Notify: New campaign for review

    PSP->>API: POST /campaigns/:id/approve
    API->>DB: Update status (approved)
    API->>DB: Generate initial tasks
    API->>BA: Notify: Campaign approved
```

Deployment Architecture

```
graph TB
    subgraph "Production Environment"
        subgraph "Container Orchestration"
            API1["API Pod 1"]
            API2["API Pod 2"]
            API3["API Pod 3"]
            Worker["Background Worker"]
        end

        subgraph "Managed Services"
            PG[(PostgreSQL<br/>Primary)]
            PGR[(PostgreSQL<br/>Read Replica)]
            Redis[(Redis Cluster)]
            S3["Object Storage"]
        end

        LB["Load Balancer"]
    end

    subgraph "Edge"
        CDN["CDN"]
        WAF["WAF"]
    end

    Users((Users)) --> WAF
    WAF --> CDN
    CDN --> LB
    LB --> API1
    LB --> API2
    LB --> API3

    API1 --> PG
    API2 --> PG
    API3 --> PGR

    API1 --> Redis
    API2 --> Redis
    API3 --> Redis

    API1 --> S3
    Worker --> PG
    Worker --> Redis
```

Security Architecture

```
graph TB
    subgraph "Client"
        App["App (HTTPS)"]
    end

    subgraph "Edge Security"
        WAF["WAF"]
        DDoS["DDoS Protection"]
        Rate["Rate Limiting"]
```

```
    end

    subgraph "Application Security"
        Auth["JWT Authentication"]
        RBAC["Role-Based Access"]
        Validation["Input Validation"]
    end

    subgraph "Data Security"
        Encryption["Encryption at Rest"]
        TLS["TLS in Transit"]
        Audit["Audit Logging"]
    end

    App --> WAF
    WAF --> DDoS
    DDoS --> Rate
    Rate --> Auth
    Auth --> RBAC
    RBAC --> Validation
    Validation --> Encryption
    Validation --> TLS
    Validation --> Audit
```

Authentication Flow

```
sequenceDiagram
    participant User
    participant App
    participant API
    participant DB
    participant Redis

    User->>App: Login (email/password)
    App->>API: POST /auth/login
    API->>DB: Validate credentials
    DB-->>API: User record

    API->>API: Generate JWT (access + refresh)
    API->>Redis: Store refresh token
    API-->>App: Tokens + user data

    App->>App: Store tokens securely

    Note over App,API: Subsequent requests

    App->>API: Request + Bearer token
    API->>API: Validate JWT
    API-->>App: Protected resource

    Note over App,API: Token refresh

    App->>API: POST /auth/refresh
    API->>Redis: Validate refresh token
    API->>API: Issue new access token
    API-->>App: New access token
```

Integration Points

External Service Integrations

Service	Purpose	Integration Method
AWS S3 / GCS	Photo storage	SDK / Signed URLs
Firebase Cloud Messaging	Push notifications	REST API
Twilio	SMS notifications	REST API
SendGrid	Email notifications	REST API
Google Maps	Geocoding, directions	REST API
Stripe	Payment processing	SDK

Internal Service Communication

```
graph LR
    subgraph "Synchronous"
        API["API Server"]
        Auth["Auth Service"]
        API -->|HTTP| Auth
    end

    subgraph "Asynchronous"
        Queue["Redis Queue"]
        Worker["Background Worker"]
        API -->|Enqueue| Queue
        Queue -->|Dequeue| Worker
    end

    subgraph "Real-time"
        WS["WebSocket Server"]
        Pub["Redis Pub/Sub"]
        API -->|Publish| Pub
        Pub -->|Subscribe| WS
    end
```

Scalability Considerations

Horizontal Scaling

Component	Scaling Strategy
API Servers	Kubernetes HPA based on CPU/requests

Background Workers	Queue depth-based scaling
PostgreSQL Reads	Read replicas
Redis	Redis Cluster
Static Assets	CDN with edge caching

Caching Strategy

```
graph LR
    Request["Request"]
    CDN["CDN Cache<br/>(Static)"]
    Redis["Redis Cache<br/>(Dynamic)"]
    App["Application Cache<br/>(In-memory)"]
    DB[(Database)]

    Request --> CDN
    CDN -->|Miss| Redis
    Redis -->|Miss| App
    App -->|Miss| DB

    DB -->|Populate| App
    App -->|Populate| Redis
```

Cache Layer	TTL	Use Case
CDN	1 hour	Static assets, public API responses
Redis	5-15 min	Session data, frequently accessed queries
Application	1 min	Hot data, computed values

Last Updated: 2026-01-01

Current Sprint: S0 Foundation

Sprint Overview

Field	Value
Sprint	S0 - Foundation
Status	**NOT STARTED**
Start Date	2026-01-06
End Date	2026-01-17
Duration	2 weeks
Team Capacity	40 SP
Committed	29 SP
Completed	0 SP

Sprint Goal

> Establish the core development infrastructure including Frappe environment, Docker configuration, ERPNext initialization, and CI/CD pipeline to enable rapid feature development in subsequent sprints.

Sprint Board

Backlog (Ready for Sprint)

ID	Title	Priority	Estimate	Assignee

To Do (WIP: 5)

ID	Title	Priority	Estimate	Assignee
POP-001	Initialize Turborepo Monorepo Structure	P0	3 SP	TBD
POP-002	Configure Docker Infrastructure	P0	5 SP	TBD
POP-003	Setup Fastify Backend & Drizzle ORM	P0	5 SP	TBD
POP-004	Define Core Database Schema	P0	8 SP	TBD

POP-005	Initialize Next.js Monorepo Apps	P1	5 SP	TBD
POP-006	Setup CI/CD Pipeline	P1	3 SP	TBD

In Progress (WIP: 0)

ID	Title	Priority	Estimate	Assignee	Started

Review (WIP: 0)

ID	Title	Priority	Estimate	Assignee	Reviewer

Done

ID	Title	Priority	Estimate	Completed

Sprint Backlog Details

POP-001: Initialize Turborepo Monorepo Structure

Priority: P0 | **Estimate**: 3 SP | **Status**: To Do

Description: Initialize Turborepo with pnpm workspaces for the new custom architecture.

Technical Details:

- Apps: `mobile-pwa`, `brand-admin`, `psp-admin`, `store-portal`
- Packages: `ui`, `api-client`, `types`, `utils`
- Backend: `api-server` (Fastify)

Acceptance Criteria:

- [ ] Turborepo initialized
- [ ] Apps and packages directories created
- [ ] Build pipeline configured
- [ ] Shared Drizzle config setup

Dependencies: None

POP-002: Configure Docker Infrastructure

Priority: P0 | **Estimate**: 5 SP | **Status**: To Do

Description: Setup Docker Compose for local development with Postgres, Redis, and API.

Technical Details:

- PostgreSQL 16 container
- Redis 7 container
- Adminer or PGAdmin (optional)
- Volume persistence

Acceptance Criteria:

- [ ] docker-compose.yml created
- [ ] Postgres connected and accessible
- [ ] Redis connected
- [ ] Environment variables configured (.env)

Dependencies: None

POP-003: Setup Fastify Backend & Drizzle ORM

Priority: P0 | **Estimate**: 5 SP | **Status**: To Do

Description: Initialize the Fastify backend service with Drizzle ORM connection.

Technical Details:

- Fastify 4.x setup
- Drizzle ORM with Postgres
- Zod validation integration
- Swagger/OpenAPI setup

Acceptance Criteria:

- [ ] Fastify server starts
- [ ] Drizzle connects to Docker Postgres
- [ ] Health check endpoint works
- [ ] Migration script functional

Dependencies: POP-001, POP-002

POP-004: Define Core Database Schema

Priority: P0 | **Estimate**: 8 SP | **Status**: To Do

Description: Implement initial Drizzle schema for Users, Auth, and Core entities.

Technical Details:

- Users table
- Auth/Session tables
- Organizations/Tenants
- Stores table

Acceptance Criteria:

- [ ] Schema defined in TypeScript
- [ ] Migrations generated
- [ ] Migrations applied successfully
- [ ] ERD generated or verified

Dependencies: POP-003

POP-005: Initialize Next.js Monorepo Apps

Priority: P1 | **Estimate**: 8 SP | **Status**: To Do

Description: Initialize Next.js applications for Brand, PSP, and Store portals within the monorepo.

Technical Details:

- `apps/brand-portal` (Next.js 14+)
- `apps/psp-portal` (Next.js 14+)
- `apps/store-portal` (Next.js 14+)
- Configure shared UI package consumption

Acceptance Criteria:

- [ ] Apps initialized successfully
- [ ] Development servers run in parallel
- [ ] Shared components render in all apps
- [ ] Build pipeline succeeds for all apps

Dependencies: POP-001

POP-006: Setup CI/CD Pipeline

Priority: P1 | **Estimate**: 3 SP | **Status**: To Do

Description: Configure GitHub Actions for linting, testing, and Docker build verification.

Acceptance Criteria:

- [ ] CI workflow created
- [ ] Lint/Type-check passes
- [ ] Unit tests execution
- [ ] Docker build verification

Dependencies: POP-001

Daily Standup Notes

Day 1 (2026-01-06)

- Sprint not yet started

Day 2 (2026-01-07)

-

Day 3 (2026-01-08)

-

Day 4 (2026-01-09)

-

Day 5 (2026-01-10)

-

Day 6 (2026-01-13)

-

Day 7 (2026-01-14)

-

Day 8 (2026-01-15)

-

Day 9 (2026-01-16)

-

Day 10 (2026-01-17)

- Sprint Review & Retrospective

Sprint Metrics

Metric	Value
Total Story Points	29 SP
Completed Points	0 SP
Remaining Points	29 SP
Sprint Progress	0%
Burndown On Track	N/A

Blockers & Risks

ID	Description	Impact	Owner	Status

Sprint Notes

- Sprint planning scheduled for 2026-01-06
- Daily standups at 9:00 AM
- Sprint review scheduled for 2026-01-17

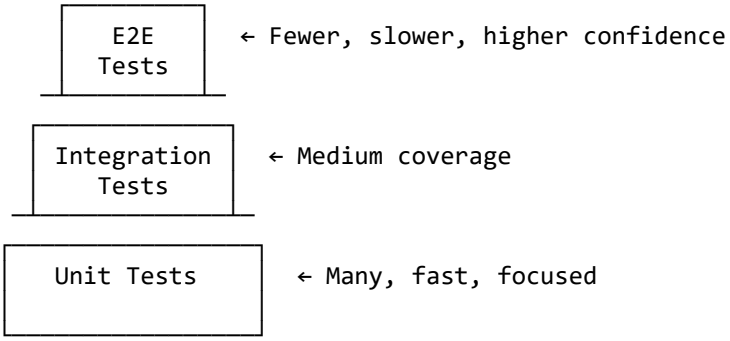
Last Updated: 2026-01-01

Testing Strategy

Overview

This document outlines the testing approach for the project, including test types, tools, coverage targets, and best practices. Our goal is to maintain high quality through a balanced testing pyramid.

Testing Pyramid



Unit Tests

Tool: Vitest

Why Vitest?

- Fast execution with native ESM support
- Jest-compatible API for easy migration
- Built-in TypeScript support
- Excellent DX with watch mode and UI

Coverage Target: 80%

Metric	Target	Minimum
Line Coverage	80%	70%
Branch Coverage	80%	70%
Function Coverage	80%	75%
Statement Coverage	80%	70%

Configuration

```
// vitest.config.ts
import { defineConfig } from 'vitest/config'

export default defineConfig({
  test: {
    globals: true,
    environment: 'jsdom',
    coverage: {
      provider: 'v8',
      reporter: ['text', 'json', 'html'],
      thresholds: {
        lines: 80,
        branches: 80,
        functions: 80,
        statements: 80
      },
      exclude: [
        'node_modules/',
        'tests/',
        '**/*.d.ts',
        '**/*.config.*'
      ]
    }
  }
})
```

Best Practices

- **Test one thing per test** - Each test should verify a single behavior
- **Use descriptive names** - `describe('UserService.createUser')` + `it('should throw error when email is invalid')`
- **Arrange-Act-Assert** - Structure tests clearly
- **Mock external dependencies** - Isolate the unit under test
- **Avoid testing implementation details** - Focus on inputs and outputs

Example

```
import { describe, it, expect, vi } from 'vitest'
import { UserService } from './user.service'

describe('UserService', () => {
  describe('createUser', () => {
    it('should create a user with valid data', async () => {
      // Arrange
      const userData = { email: 'test@example.com', name: 'Test User' }
      const mockRepo = { save: vi.fn().mockResolvedValue({ id: 1, ...userDat
      const service = new UserService(mockRepo)

      // Act
      const result = await service.createUser(userData)
```

```
      // Assert
      expect(result.id).toBe(1)
      expect(mockRepo.save).toHaveBeenCalledWith(userData)
    })

    it('should throw error when email is invalid', async () => {
      // Arrange
      const userData = { email: 'invalid', name: 'Test User' }
      const service = new UserService({})

      // Act & Assert
      await expect(service.createUser(userData)).rejects.toThrow('Invalid em
    })
  })
})
```

Integration Tests

Focus: API Endpoint Testing

Integration tests verify that different parts of the system work together correctly, with a focus on API endpoints and service integrations.

What to Test

- **API endpoints** - Request/response validation
- **Database operations** - CRUD operations with real database
- **External service integrations** - Third-party API calls (mocked)
- **Authentication/authorization** - Security flows

Tools

- **Vitest** - Test runner
- **Supertest** - HTTP assertions
- **Test containers** - Isolated database instances

Configuration

```
// integration.config.ts
import { defineConfig } from 'vitest/config'

export default defineConfig({
  test: {
    include: ['tests/integration/**/*.test.ts'],
    setupFiles: ['tests/integration/setup.ts'],
    testTimeout: 30000,
    hookTimeout: 30000
  }
})
```

Example

```
import { describe, it, expect, beforeAll, afterAll } from 'vitest'
import request from 'supertest'
import { app } from '../src/app'
import { setupTestDatabase, teardownTestDatabase } from './helpers'

describe('POST /api/orders', () => {
  beforeAll(async () => {
    await setupTestDatabase()
  })

  afterAll(async () => {
    await teardownTestDatabase()
  })

  it('should create an order and return 201', async () => {
    const orderData = {
      customerId: 1,
      items: [{ sku: 'ABC123', quantity: 2 }]
    }

    const response = await request(app)
      .post('/api/orders')
      .set('Authorization', 'Bearer valid-token')
      .send(orderData)

    expect(response.status).toBe(201)
    expect(response.body.orderId).toBeDefined()
    expect(response.headers.location).toMatch(/\/api\/orders\/\d+/)
  })

  it('should return 400 for invalid order data', async () => {
    const response = await request(app)
      .post('/api/orders')
      .set('Authorization', 'Bearer valid-token')
      .send({})

    expect(response.status).toBe(400)
    expect(response.body.error).toBe('VALIDATION_ERROR')
  })
})
```

E2E Tests

Tool: Playwright

Why Playwright?

- Cross-browser support (Chromium, Firefox, WebKit)
- Auto-wait capabilities reduce flakiness
- Powerful debugging with trace viewer
- Parallel execution for speed

Configuration

```
// playwright.config.ts
import { defineConfig, devices } from '@playwright/test'

export default defineConfig({
  testDir: './tests/e2e',
  timeout: 30000,
  retries: 2,
  workers: 4,
  reporter: [['html'], ['junit', { outputFile: 'results.xml' }]],
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure'
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
    { name: 'mobile', use: { ...devices['iPhone 13'] } }
  ]
})
```

Critical User Flows to Test

1. **User Registration & Login**
2. **Product Search & Browse**
3. **Add to Cart & Checkout**
4. **Order Management**
5. **Account Settings**

Example

```
import { test, expect } from '@playwright/test'

test.describe('Checkout Flow', () => {
  test('user can complete purchase', async ({ page }) => {
    // Login
    await page.goto('/login')
    await page.fill('[data-testid="email"]', 'test@example.com')
    await page.fill('[data-testid="password"]', 'password123')
    await page.click('[data-testid="login-button"]')

    // Add item to cart
    await page.goto('/products/123')
    await page.click('[data-testid="add-to-cart"]')

    // Checkout
    await page.goto('/cart')
    await page.click('[data-testid="checkout-button"]')

    // Fill shipping info
```

```
    await page.fill('[data-testid="address"]', '123 Test St')
    await page.click('[data-testid="continue-to-payment"]')

    // Complete order
    await page.click('[data-testid="place-order"]')

    // Verify success
    await expect(page.locator('[data-testid="order-confirmation"]')).toBeVis
    await expect(page.locator('[data-testid="order-number"]')).toHaveText(/C
  })
})
```

Manual Testing

Exploratory Testing

Exploratory testing complements automated tests by finding issues that scripted tests might miss.

When to Perform:

- Before major releases
- After significant feature additions
- When investigating reported issues

Focus Areas:

- Edge cases and unusual workflows
- Error handling and recovery
- Performance under stress
- Usability and user experience

Documentation:

- Use session-based test management
- Document findings immediately
- Create bugs or test cases from discoveries

Accessibility Testing

Ensure WCAG 2.1 AA compliance for all user-facing features.

Tools:

- **axe DevTools** - Browser extension for automated checks
- **WAVE** - Web accessibility evaluation tool
- **Screen readers** - NVDA (Windows), VoiceOver (Mac)

Checklist:

- [ ] Keyboard navigation works for all interactive elements
- [ ] Focus indicators are visible
- [ ] Color contrast meets 4.5:1 ratio
- [ ] Images have alt text
- [ ] Forms have proper labels
- [ ] Error messages are announced to screen readers
- [ ] Page structure uses semantic HTML

Test Execution Schedule

Test Type	When to Run	Duration
Unit Tests	Every commit, PR	~1-2 min
Integration Tests	Every PR	~5-10 min
E2E Tests (smoke)	Every PR	~10-15 min
E2E Tests (full)	Nightly, pre-release	~30-60 min
Manual/Exploratory	Weekly, pre-release	2-4 hours
Accessibility	Per feature, pre-release	1-2 hours

Reporting & Metrics

Key Metrics

- Test pass rate
- Code coverage percentage
- Test execution time
- Flaky test rate

CI Integration

- Tests run on every pull request
- Coverage reports published as PR comments
- Test failures block merge

Last Updated: 2026-01-01

Definition of Done (DoD)

Overview

The Definition of Done is a shared understanding of what it means for a user story or task to be considered complete. All items must be checked before a story can be moved to "Done."

Quality Criteria Checklist

Code Quality

- [ ] **Code complete and committed** - All code changes are finalized and pushed to the feature branch
- [ ] **Code follows project coding standards** - Linting passes with no errors
- [ ] **No hardcoded values** - Configuration externalized appropriately

Testing

- [ ] **Unit tests passing** - All existing and new unit tests pass
- [ ] **Code coverage ≥80%** - Minimum coverage threshold met for new code
- [ ] **No test flakiness** - Tests are reliable and deterministic

Code Review

- [ ] **Code reviewed and approved** - At least one team member has reviewed and approved the PR
- [ ] **Review comments addressed** - All feedback has been resolved or discussed

Documentation

- [ ] **Documentation updated** - README, API docs, or inline comments updated as needed
- [ ] **Changelog updated** - Notable changes recorded for release notes

Bug Status

- [ ] **No P0/P1 bugs** - No critical or high-priority bugs remain open for this feature
- [ ] **Known issues documented** - Any accepted limitations are documented

Deployment

- [ ] **Deployed to staging** - Changes successfully deployed to staging environment
- [ ] **Smoke tests passing** - Basic functionality verified in staging

Acceptance

- [ ] **Acceptance criteria verified** - All acceptance criteria from the user story are met
- [ ] **Product Owner sign-off** - PO has reviewed and accepted the implementation

Accessibility

- [ ] **Accessibility checked (WCAG 2.1 AA)** - UI changes meet accessibility standards
- [ ] **Keyboard navigation works** - All interactive elements accessible via keyboard
- [ ] **Screen reader compatible** - Content readable by assistive technologies

---

Quick Reference

Category	Criteria	Required
Code	Complete and committed	✅
Testing	Unit tests passing (≥80% coverage)	✅
Review	Code reviewed and approved	✅
Docs	Documentation updated	✅
Bugs	No P0/P1 bugs	✅
Deploy	Deployed to staging	✅
Accept	Acceptance criteria verified	✅
A11y	WCAG 2.1 AA compliance	✅

Exceptions

In rare circumstances, exceptions may be granted by the Tech Lead or Product Owner. All exceptions must be:

1. Documented in the ticket
2. Tracked as technical debt if applicable
3. Addressed within the next sprint

Last Updated: 2026-01-01